

IDA-VIRUMAA KUTSEHARIDUSKESKUS

Maksim Dzjubenko, Maksim Grišin

JPTV20

SOTSIAALVÕRGUSTIK „SKILL SHUFFLE“

Lõputöö

Juhendaja: Jüri Melnikov

Jõhvi 2024

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1. Техническое задание проекта	4
1.1. Постановка задачи	4
1.2. SWOT-анализ	5
2. Планирование проекта.....	6
2.1. Члены команды	6
2.2. Требования	6
2.3. Пользовательские сценарии.....	6
2.4. Дизайн и стилистика проекта	11
2.5. Бюджет и планирование времени.....	13
2.6. Управление рисками.....	14
3. Развитие проекта	16
3.1. Инструменты разработки	16
3.2. Frontend разработка	16
3.3. Backend разработка.....	50
заключение	65
источники	Ошибка! Закладка не определена.
4. Приложения	67
4.1. Список иллюстраций.....	67

ВВЕДЕНИЕ

Социальные сети стали неотъемлемой частью нашей повседневной жизни, они объединяют людей со всего мира на основе общих интересов, целей или деятельности. В современном мире, где навыки и компетенции играют все более важную роль в карьерном и личностном развитии, существует потребность в создании платформы, которая позволит людям обмениваться знаниями, навыками и опытом.

Цель данной работы заключается в создании проекта социальной сети под названием "Skill Shuffle". Этот проект направлен на создание онлайн-платформы, которая объединяет людей на основе общих интересов, деятельности или связей, обмена опытом и совместного изучения, позволит пользователям обмениваться навыками, учиться у других пользователей, находить единомышленников и развиваться как профессионалы в своей области.

Зарегистрировавшимся пользователям доступны через боковое меню в левой части экрана чат с фильтрами, профиль пользователя, новостная лента на основе подписок как главная страница после входа, поиск пользователей по имени или тегу, создание записи для своего профиля, список уведомлений разного характера, а также меню с выпадающим окном, через которое можно совершить выход из своего аккаунта, быстро перейти к сохраненным вкладкам, сообщить об ошибке в поддержку сайта.

На странице профиля можно просматривать лично выставленные и понравившиеся публикации, сохраненные вкладки и комментарии к ним. Через страницу профиля доступен просмотр друзей, подписчиков, запросов в друзья, сообществ, фото и видео, настроек профиля.

Выпускную работу писали два автора, которые совместно работали над всеми этапами проекта. Вся разработка велась с использованием облака AWS. Авторы использовали для разработки проекта React.js и Spring Boot. Выбор языков программирования обусловлен навыками владения для написания проекта такого масштаба.

Период выполнения дипломной работы: 15.03.24 – 17.05.24.

GitHub: <https://github.com/mak7ilenin/Skill-Shuffle>

URL: <https://skillshuffle.ddns.net>

1. ТЕХНИЧЕСКОЕ ЗАДАНИЕ ПРОЕКТА

1.1. Постановка задачи

Цель данной работы заключается в создании и анализе проекта социальной сети под названием "Skill Shuffle". Этот проект направлен на создание онлайн-платформы, которая позволит пользователям обмениваться навыками, учиться у других пользователей, находить единомышленников и развиваться как профессионалы в своей области.

Этапы разработки проекта:

Анализ и планирование:

1. Провести анализ требований и целей проекта;
2. Определить основные функциональные возможности платформы;
3. Разработать техническое задание.

Front-End:

Создать дизайн и прототип интерфейса в Figma.

Разработать пользовательские страницы:

1. Главная страница с новостной лентой на основе подписок;
2. Страницы профилей пользователей, настройки профиля пользователя;
3. Страница поиска пользователей;
4. Страница мессенджера;
5. Страница уведомлений;
6. Страница сообществ;
7. Страница создания публикации.

Back-End:

1. Разработать серверную часть;
2. Аутентификация и авторизация пользователей;
3. Логика работы с данными (записи, комментарии, профили);
4. Взаимодействие с базой данных.

Инфраструктура:

1. Развернуть облако (например, AWS);
2. Настроить доменное имя и SSL-сертификат;
3. Обеспечить масштабируемость и безопасность.

Тестирование:

1. Провести модульное и интеграционное тестирование;
2. Проверить работоспособность всех функций.

Публикация и поддержка:

1. Развернуть проект на сервере;
2. Поддерживать и обновлять платформу.

1.2. SWOT-анализ

Для четкого понимания слабых и сильных сторон, возможностей и опасностей реализации проекта авторы обозначили их при помощи SWOT-анализа.

Сильные стороны	Слабые стороны
1. Уникальная идея: “Skill Shuffle” предлагает уникальную платформу для обмена навыками и опытом между пользователями; 2. Обширная база пользователей: Социальная сеть объединяет людей на основе общих интересов и деятельности; 3. Использование облачных технологий AWS: это обеспечивает масштабируемость и надежность системы.	1. Недостаточная реклама: Проект может столкнуться с проблемой привлечения пользователей из-за отсутствия рекламы; 2. Ограниченные ресурсы: Недостаточное количество сотрудников может замедлить развитие проекта.
Возможности	Опасности
1. Расширение функционала: в “Skill Shuffle” можно добавить новые функции, такие как онлайн-курсы или виртуальные мероприятия; 2. Партнерство с образовательными учреждениями: Сотрудничество с университетами или школами может привлечь больше пользователей.	1. Конкуренция: существует множество других социальных сетей, и “Skill Shuffle” должен бороться за свою аудиторию; 2. Технические проблемы: неполадки в облачной инфраструктуре или безопасности могут повлиять на работу платформы.

Таблица 1. SWOT-анализ

2. ПЛАНИРОВАНИЕ ПРОЕКТА

2.1. Члены команды

Команда состояла из двух человек. Оба автора являются full-stack разработчиками и дизайнерами проекта, совместно выполняющие все части работы.

2.2. Требования

Все требования к проекту заключались в создании удобной, доступной и многофункциональной социальной сети, которая легко дается новым пользователям на уровне интуиции. Важно также сказать, что дизайн и структура не должны быть перегружены информацией, но в тот же момент являться полноценными.

2.3. Пользовательские сценарии

В проекте существует единственная роль пользователей – user. Весь функционал сайта доступен после регистрации пользователя на сайте. Гостю, не создавшему аккаунт, доступны только регистрация на сайт.

№	Сценарий	Описание
1	Регистрация нового пользователя	1. Пользователь открывает сайт и нажимает на кнопку “Sign up”; 2. Система предлагает ввести: • почту; • логин; • пароль; • имя и фамилию; • „nickname“ (работает как тег); • другие обходимые либо необходимые данные. 3. Пользователь заполняет форму и нажимает “Зарегистрироваться”; 4. Система создает новый аккаунт и показывает кнопку, помогающую перенаправить пользователя на страницу входа, если данные введены в правильном формате.

№	Сценарий	Описание
2	Вход пользователя	Пользователь заполняет форму входа (имя пользователя и пароль) и нажимает кнопку „SIGN IN“, после чего система сверяет введенные данные с данными базы данных. Если данные неверны, то пользователь получает текст с ошибкой. В случае если все данные верны, пользователь успешно заходит в систему и будет иметь доступ к сайту 2 часа, если при входе пользователь не нажимал кнопку «запомнить меня», если же кнопка была нажата, то пользователь может оставаться в системе 7 дней, не выходя из неё.
3	Просмотр новостной ленты	1. На главной странице пользователь видит список записей от пользователей и сообществ, на которые он подписан (так же перейти на главную страницу можно по кнопке в левом меню); 2. Пользователь может просматривать, комментировать, ставить отметку-лайк, делиться записями, а также делать закладки.
4	Создание записи	1. Пользователь нажимает в левом меню кнопку “Создать запись”; 2. Вводит текст записи и прикрепляет фото, видео, файлы (если нужно), выбирает вид публичности записи (общедоступный или только для друзей), настраивает возможность комментирования и авто отправки уведомлений подписавшимся пользователям; 3. Нажимает “Опубликовать”.
5	Поиск других пользователей	1. Пользователь переходит на страницу поиска по кнопке в левом меню; 2. Вводит имя или тег другого пользователя; 3. Система выводит результаты поиска.
6	Общение в мессенджере	1. Пользователь выбирает мессенджер из бокового меню;

№	Сценарий	Описание
		2. Применяет фильтры, если нужно. Например, тип чата, такие как контакты, групповые или сообщества. Или же поиск по названию чата; 3. Общается в чатах другими пользователями, отправляет GIF, эмодзи, фото и видео (в том числе и загруженные лично в профиль или напрямую с устройства в чат), файлы.
7	Управление меню чата, группы или сообщества	1. Пользователь выбирает мессенджер из бокового меню; 2. Выбирает нужный чат с пользователем, группой или сообществом; 3. Нажимает кнопку меню чата; 4. Выбирает одну из функций: • Отключить уведомления; • Очистить историю сообщений; • Выйти из чата.
8	Создание чата и группы	1. Пользователь запускает мессенджер; 2. Нажимает кнопку создания рядом с поиском; 3. Выбирает одного или нескольких пользователей, являющимися друзьями для пользователя, выполнившего вход в приложение; 4. Добавляет картинку и название чату (если нужно); 5. В зависимости от выборов будет создаваться либо личный чат, либо группа. Если добавлена картинка, название или несколько пользователей, то создается группа. В остальных случаях – личный чат; 6. Пользователь подтверждает создание.
9	Управление группой	1. Пользователь открывает любой групповой чат и нажимает на название чата в верхнем блоке; 2. В левом меню пользователь может посмотреть: • название группы; • количество участников; • картинку группы;

№	Сценарий	Описание
		• краткую информацию о каждом участнике – полное имя, последняя активность, изображение пользователя, роль (роль видна только у администраторов чата, у остальных пользователей роль участника); 3. Пользователь может использовать поиск участников по имени по нажатию на лупу; 4. Добавить своего друга/друзей в текущий чат (если это позволено в настройках чата); 5. Также пользователь может выйти из группы и больше не получать сообщений из него; 6. Создатель группы может удалять участников или повышать их до роли администратора по стрелочке рядом с каждым участником.
10	Просмотр профиля	1. Пользователь переходит на страницу своего профиля в левом меню; 2. Может просматривать: • свои публикации; • понравившиеся публикации; • сохраненные вкладки; • комментарии под публикацией; • список фотографий и видео; • список комментариев. Также доступен просмотр: • друзей (также друзей в сети); • подписчиков; • людей по общим признакам; • основных данных пользователя; • баннера и настроек профиля. 3. Через меню категорий быстрого доступа можно просматривать, отслеживать и редактировать категории: • друзья:

№	Сценарий	Описание
		▪ все друзья; ▪ друзья онлайн; • подписчики: ▪ подписчики; ▪ подписки на людей; • запросы в друзья; ▪ входящие; ▪ исходящие; • сообщества; ▪ все сообщества; ▪ управляемые сообщества; • фото и видео; • заблокированные пользователи.
11	Редактирование профиля	1. Пользователь переходит на страницу своего профиля; 2. Нажимает кнопку „Edit profile“; 3. Доступно редактирование нескольких категорий профиля, такие как: • Персональная информация: ▪ имя; ▪ фамилия; ▪ дата рождения; ▪ пол; ▪ биография; ▪ „nickname“. • Конфиденциальность: ▪ публикации; ▪ профиль. • Уведомления: ▪ отметки „нравится“; ▪ комментарии; ▪ репосты; ▪ личные сообщения; ▪ запросы в друзья;

№	Сценарий	Описание
		▪ упоминания; ▪ подписчики. • Безопасность: ▪ почта; ▪ логин и пароль; ▪ двухфакторная аутентификация; ▪ удаление аккаунта; 4. Подтверждает изменения.
12	Создание сообщества	1. После входа на сайт пользователь переходит на страницу профиля; 2. Нажимает на ссылку «communities»; 3. Нажимает на кнопку «Create community»; 4. Вводит название, тег и вид приватности сообщества; 5. Выбирает тематику; 6. Далее по желанию устанавливает фото сообществу, отправляет приглашения своим друзьям, делает описание сообщества заканчивает создание.

Таблица 2. Пользовательские сценарии

2.4. Дизайн и стилистика проекта

Название сайта было основано на основной цели проекта. Логотип сайта состоит из двух латинских букв “S”. Шрифт названия логотипа “Russo One” (см. Рисунок 1).

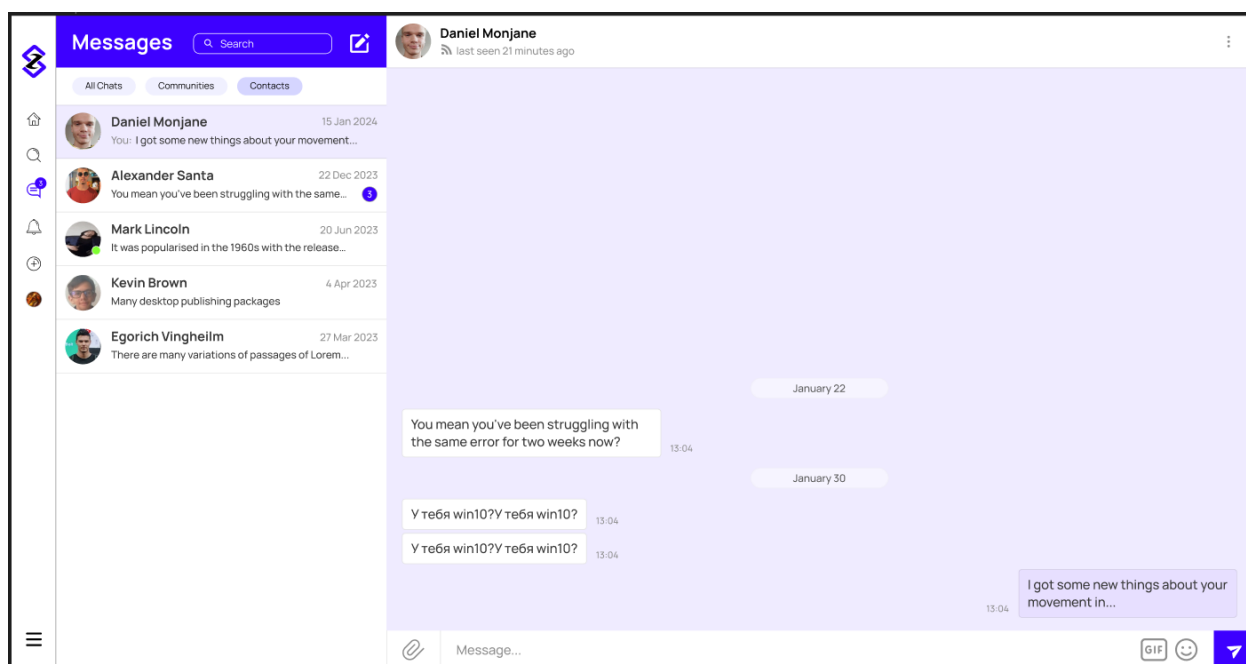
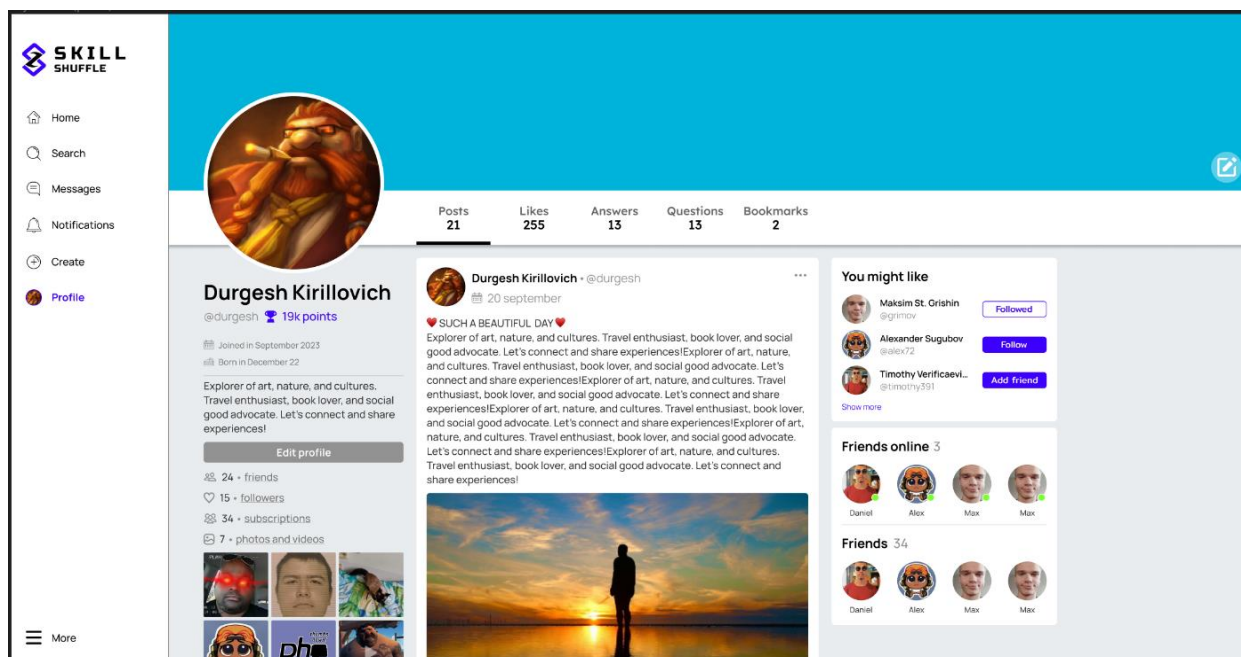


Основной используемый шрифт для текста был “Manrope”. Второстепенный шрифт (например, для формы входа) был “Mitr”.

Основные цвета, используемые в дизайне сайта, были серый и синий (см Рисунок 2).



Для разработки дизайна и прототипа авторы использовали Figma. Figma – это online-сервис для разработки дизайна и прототипа веб-сайта с большим количеством расширений и возможностью совместной работы. Конечный внешний вид нескольких основных страниц в Figma выглядит так (см Рисунок 3 и 4)



2.5. Бюджет и планирование времени

Проект реализовывался с 20.01.24 по 17.05.24. Выделенное время на каждый этап работы было сведено в описывающую таблицу. Время указано суммарно на двух человек.

Этапы	Затраченное время (астрономические часы)	Предполагаемое время (астрономические часы)
Анализ и планирование	42	39
Дизайн сайта	462	306

Этапы	Затраченное время (астрономические часы)	Предполагаемое время (астрономические часы)
Front-End	224	198
Back-End	540	278
Инфраструктура	6	6
Тестирование	61	60
Запуск и поддержка	232	120
Общее время	1567	1007

Таблица 3. Затраченное время на этапы работы

Оплата труда рассчитана на основе средней нетто-ставки оплаты труда веб-разработчиков в Эстонии (приблизительно €11/час).

Использовались сервисы Amazon с бесплатным периодом пользования, такие как:

1. Elastic Compute Cloud [1];
2. Relational Database Service [2];
3. Elastic Container Registry [3];
4. CloudWatch [4];
5. Cloud Object Storage [5].

Этапы	Оплата
Анализ и планирование	462€
Дизайн сайта	5082€
Front-End	2464€
Back-End	5940€
Инфраструктура	66€
Тестирование	671€
Запуск и поддержка	2552€
Общие оплата	17237€

Таблица 4. Затраты на проект

2.6. Управление рисками

Для удобства вся информация о рисках и их мерах решения была сведена в одну общую таблицу.

Риски	Меры решения
Утечка данных, хакерские атаки.	Реализовать меры безопасности, такие как шифрование данных и двухфакторную аутентификацию. - Провести полное тестирование безопасности.
Возможные проблемы с разработкой серверной части, инфраструктурой, тестированием и обеспечением безопасности.	Провести дополнительное обучение команды по необходимым технологиям. - Нанять опытных специалистов. - Разработать план бэкапа и восстановления данных.
Плохая оптимизация веб-сайта и низкая производительность	По мере разработки выделять время на рефакторинг кода, при увеличении трафика повышать производительность сервера.
Высокие денежные затраты в случае большого органического трафика на масштабируемость хранилища статических файлов, базы данных, виртуального сервера	Развёртывание приложений с помощью Docker (для более эффективного распределения ресурсов и масштабирования), мониторинг слабых мест производительности, кэширование запросов. Также можно найти возможных инвесторов.

Таблица 5. Риски и их меры решения

3. РАЗВИТИЕ ПРОЕКТА

Проект был структурирован на три основные области. Максим Дзюбенко и Максим Гришин отвечали за дизайн. Разработку интерфейса вел Максим Гришин, в то время как Максим Дзюбенко занимался серверной частью. Командные усилия были направлены на регулярное тестирование продукта на всех этапах его создания.

3.1. Инструменты разработки

Frontend разработка: Использование React.js обусловлено его эффективностью и гибкостью как JavaScript-библиотеки для создания динамичных пользовательских интерфейсов. Это особенно актуально для социальных сетей, где важно обеспечить быстрое обновление данных без перезагрузки страницы. Компоненты React-Bootstrap [6], CSS и SCSS использовались для стилизации, при этом SCSS позволил максимально эффективно работать со стилями.

Backend разработка: Java и Spring Boot [7] были выбраны для серверной части из-за их мощности и гибкости. Spring Boot подходит для социальных сетей благодаря быстрой разработке, автоконфигурации и поддержке микросервисной архитектуры. Он также облегчает интеграцию с базами данных и управление безопасностью, что критично для социальных сетей. MariaDB была выбрана в качестве системы управления базами данных.

ХАМРР использовался для локального тестирования, а Docker [8] — для оптимизации процесса развертывания. Docker гарантировал идентичность сред разработки и продакшена, уменьшая риск ошибок и ускоряя развертывание за счет контейнеризации зависимостей и настроек.

В процессе разработки бэкенда использовалась IntelliJ IDEA, а для фронтенда — Visual Studio Code. Эти инструменты повысили эффективность и качество кода. Библиотеки и плагины для Java и Spring Boot были тщательно отобраны для интеграции в проект, о чем будет рассказано в последующих разделах.

3.2. Frontend разработка

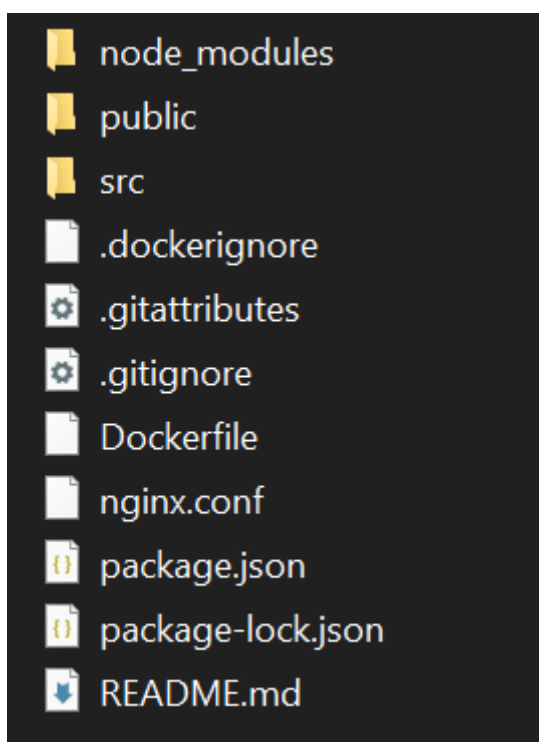
В процессе разработки клиентской части проекта социальной, разработчики, стремились создать интуитивно понятный и визуально привлекательный пользовательский интерфейс. Начальным этапом было создание детального макета в Figma, который послужил основой для дальнейшей работы и помог определить структуру и дизайн будущего приложения.

Выбор React.js в качестве основной технологии фронтенда был обусловлен его мощными возможностями по созданию динамичных и современных веб-приложений. React.js позволил нам эффективно реализовать сложные пользовательские взаимодействия и обеспечить высокую скорость работы приложения.

С учетом этих факторов команда приступила к разработке, опираясь на принципы модульности и повторного использования кода, что в итоге позволило создать гибкое и масштабируемое решение для социальной сети.

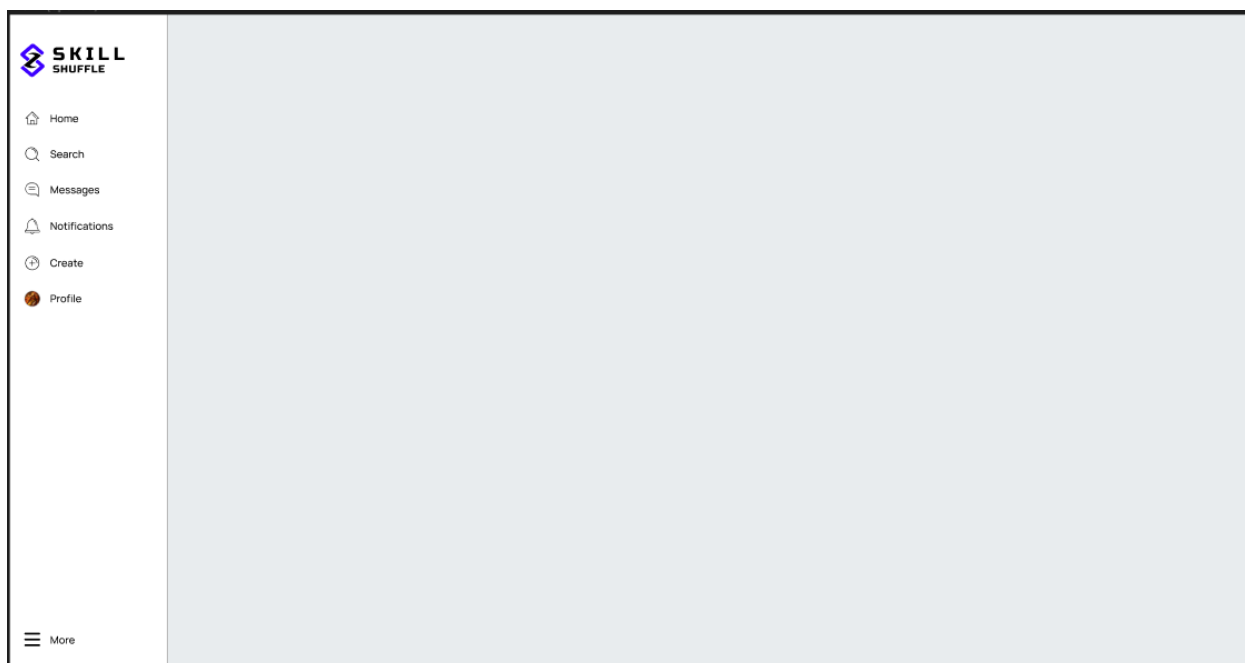
3.2.1 Создание структуры

Сначала разработчики занимались запуском React приложением и устанавливали нужные зависимости, а также составляли доступную и понятную структуру проекта. Директория клиентской части выглядит следующим образом. В папке `src` находятся файлы для разработки структуры сайта и скриптов, в папке `public` хранятся статичные файлы, папка `node_modules` хранит в себе зависимости проекта. В корневой папке хранятся файлы конфигурации (см Рисунок 5).



3.2.2 Боковое меню

Для удобного перемещения между страницами, было разработано боковое меню. Расположение меню слева кажется весьма странным и нестандартным, но на практике оно очень удобно в использовании (см Рисунок 6).



Вместе с боковым меню параллельно создавалась логика для хранения авторизованного пользователя в браузере. Поскольку cookie с токеном созданные на сервере, невозможно получить со стороны клиента, а хранить важные данные на клиентской части очень опасно в плане безопасности. Поэтому был принято решение хранить только имя, никнейм, ссылку на изображение. Но для подключения к WebSocket [9] протоколу, нужен токен для подтверждения авторизации, поэтому приходится делать запрос на сервер для проверки подлинности токена из cookie и возвращать токен обратно на клиентскую часть. Практически половина функционала связано с протоколом WebSocket, например уведомления, чаты, статус активности и другое, вот пример подключения (см. Рисунок 7 и 8).

```

import React, { createContext, useState, useEffect, useContext, useCallback } from 'react';
import axios from 'axios';
import { Client } from '@stomp/stompjs';

import { API_SERVER, WEBSOCKET_URL } from '../config';
import MessageNotification from '../MessageNotification';

const AuthContext = createContext();

const AuthProvider = ({ children }) => {
  const [stompClient, setStompClient] = useState(null);
  const [isStompClientInitialized, setIsStompClientInitialized] = useState(false);
  const [messageNotification, setMessageNotification] = useState({ visible: false, notification: {} });
  const [authUser, setAuthUser] = useState(() => {
    return JSON.parse(sessionStorage.getItem('auth-user'));
  });
};

```

```

const AuthProvider = ({ children }) => {

  // Subscribe to notifications
  const subscribeToNotifications = useCallback(() => {
    if (!isStompClientInitialized) return;

    stompClient.subscribe('/user/notification', receivedNotification => {
      // Process the received notification
      const notification = JSON.parse(receivedNotification.body);
      setMessageNotification({ visible: true, notification: notification });
    });
  }, [isStompClientInitialized, stompClient]);

  // Initialize the WebSocket connection
  const initializeStompClient = useCallback(() => {
    if (stompClient) {
      subscribeToNotifications();
      return;
    }

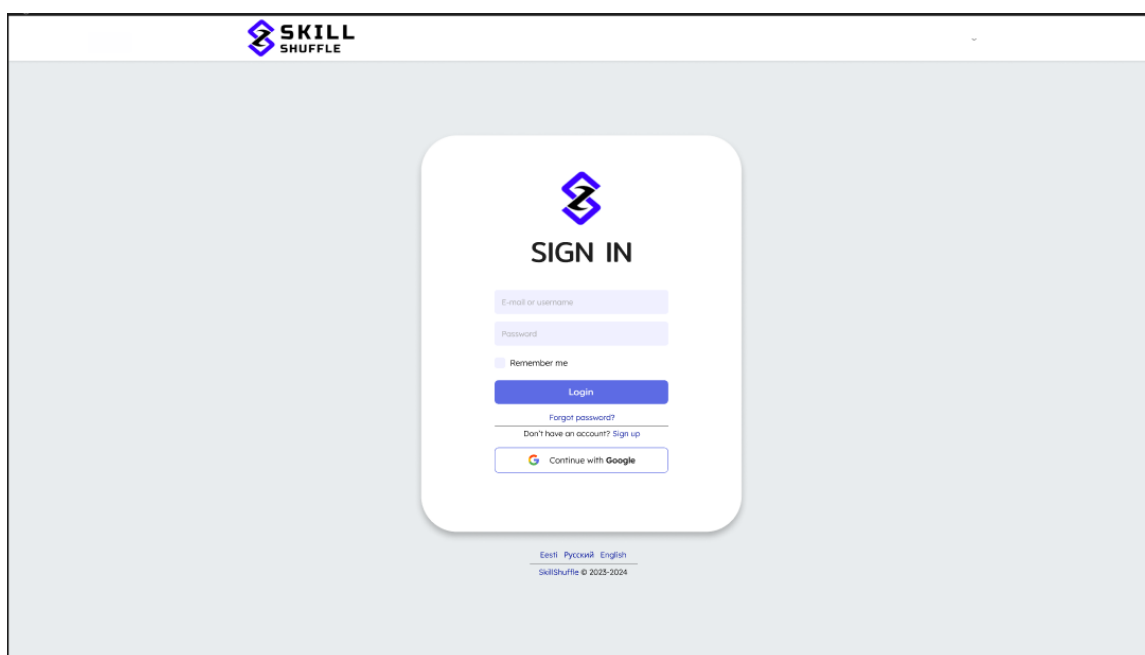
    axios.get(`${API_SERVER}/auth/confirm`, { withCredentials: true })
      .then(response => {
        // Store the user in the session storage
        setAuthUser(response.data.user);

        // Create a new WebSocket connection
        const client = new Client({
          brokerURL: WEBSOCKET_URL,
          connectHeaders: {
            Authorization: `Bearer ${response.data.accessToken}`,
          },
          onConnect: function (frame) {
            console.log('Connected: ' + frame);
            setStompClient(client);
            setIsStompClientInitialized(true);
          },
          onDisconnect: function (frame) {
            console.log('Disconnected: ' + frame);
            setStompClient(null);
            setIsStompClientInitialized(false);
          }
        });
      });
  }, [stompClient]);
};

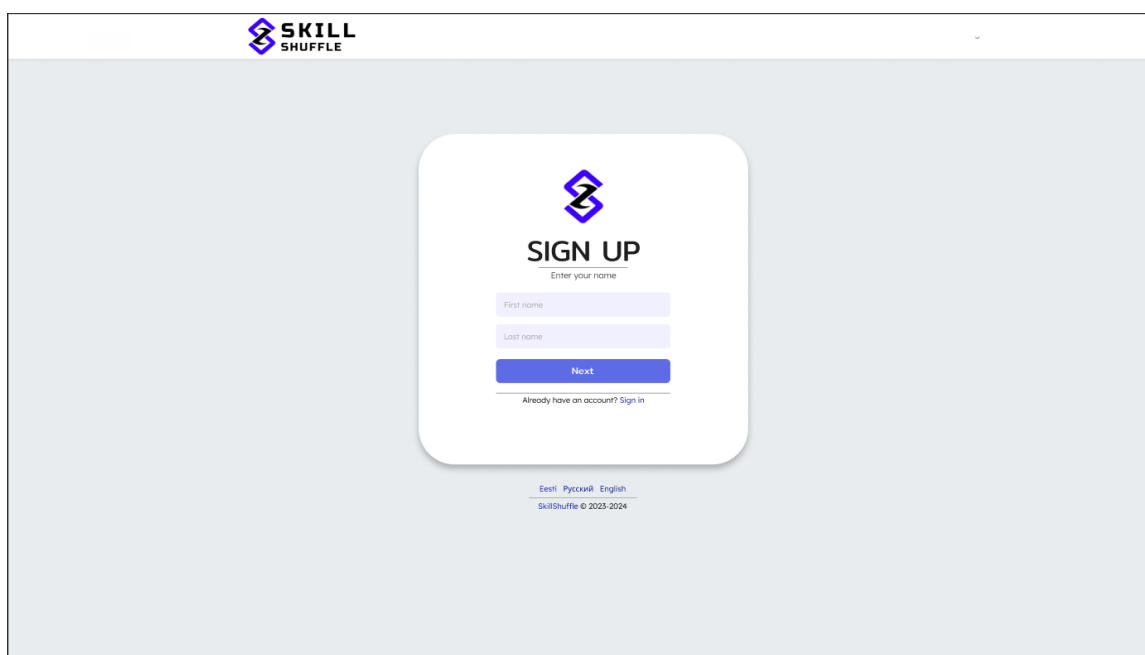
```

3.2.3 Авторизация

При загрузке сайта будет отображено окно входа, оно будет видно для неавторизованных пользователей, это единственное что пользователь может просмотреть на сайте будучи неавторизованным в системе. С помощью этой страницы можно зайти в аккаунт и в случае необходимости нажать ‘Remember me’, если пользователь хочет оставаться в системе 7 дней без надобности повторно проходить аутентификацию. Можно также зарегистрироваться, если у пользователя нет аккаунта, это можно сделать, нажав на кнопку ниже под кнопкой входа. Регистрация имеет 6 этапов для полноценного создания аккаунта (см Рисунок 9 и 10).



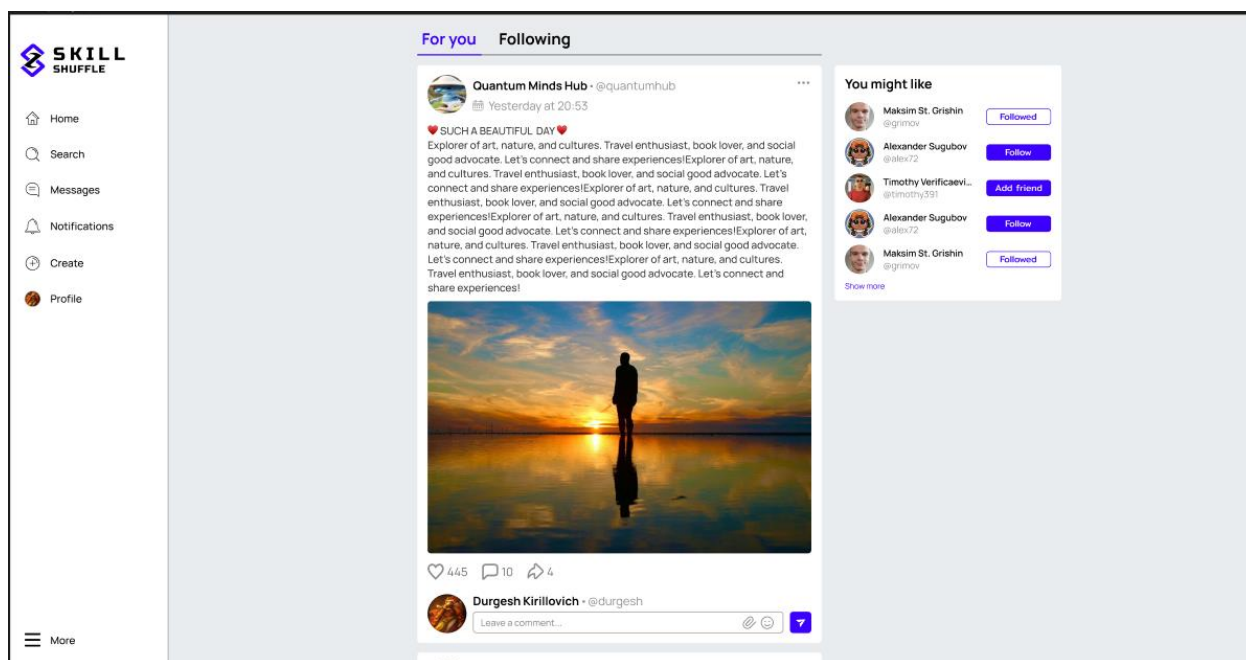
The screenshot shows the 'SIGN IN' page of the Skill Shuffle application. At the top left is the 'SKILL SHUFFLE' logo. The main content is a white rounded rectangle centered on a light gray background. Inside this rectangle, at the top, is a blue and purple geometric logo. Below it is the text 'SIGN IN'. There are two input fields: 'E-mail or username' and 'Password'. Below these is a checkbox labeled 'Remember me'. A blue 'Login' button is positioned below the checkbox. Underneath the button are two links: 'Forgot password?' and 'Don't have an account? Sign up'. At the bottom of the white rectangle is a button labeled 'Continue with Google' with the Google logo. At the very bottom of the page, there is a footer with the text 'Eesti Русский English' and 'SkillShuffle © 2023-2024'.



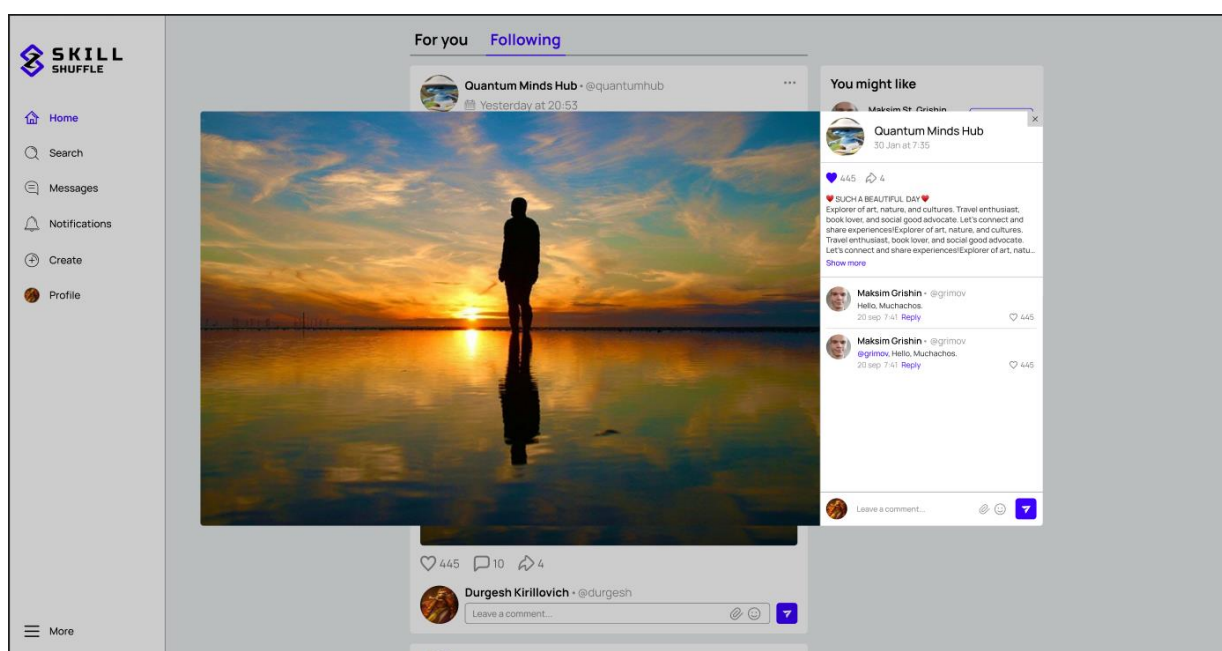
The screenshot shows the 'SIGN UP' page of the Skill Shuffle application. At the top left is the 'SKILL SHUFFLE' logo. The main content is a white rounded rectangle centered on a light gray background. Inside this rectangle, at the top, is a blue and purple geometric logo. Below it is the text 'SIGN UP' followed by the instruction 'Enter your name'. There are two input fields: 'First name' and 'Last name'. A blue 'Next' button is positioned below these fields. Below the button is a link: 'Already have an account? Sign in'. At the very bottom of the page, there is a footer with the text 'Eesti Русский English' and 'SkillShuffle © 2023-2024'.

3.2.4 Домашняя страница

После успешной авторизации пользователь попадает на домашнюю страницу с рекомендуемыми публикациями, это будут записи от подписанных пользователем друзей и сообществ. Нажав на кнопку ‘Following’, можно также просматривать опубликованные записи только от тех сообществ или друзей, на которых пользователь подписан (см Рисунок 11).

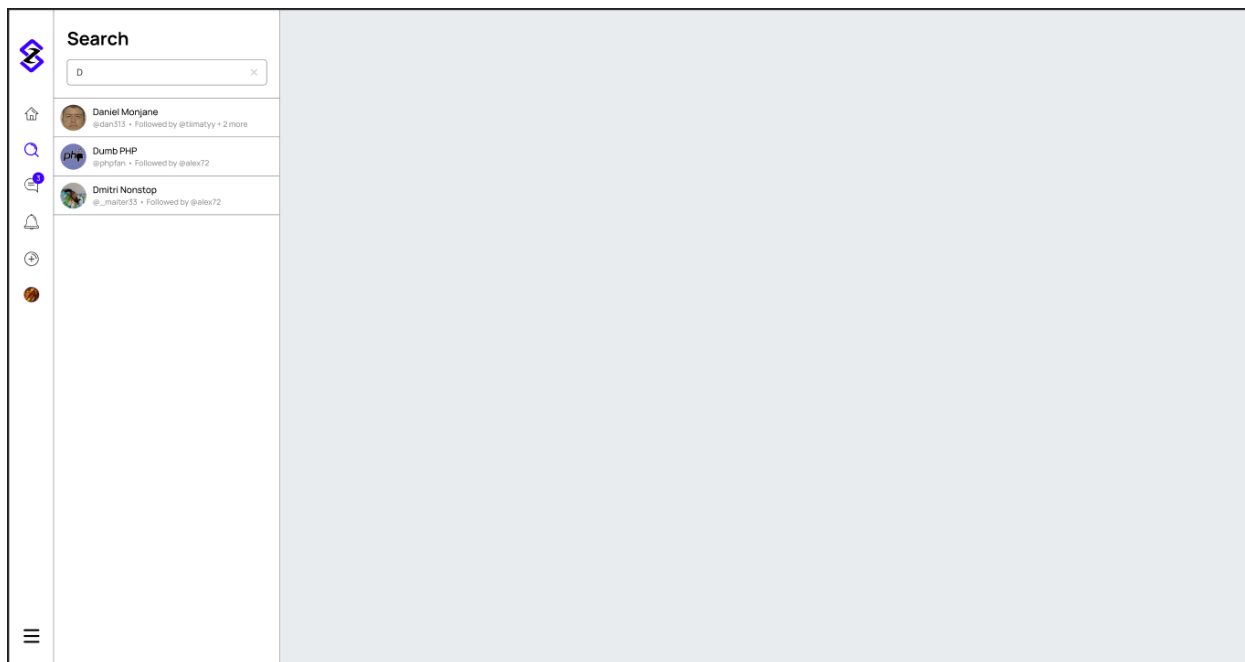


Если публикация имеет изображение, то нажав на него откроется окно с публикацией с увеличенным масштабом картинки для удобного просмотра (см Рисунок 12).



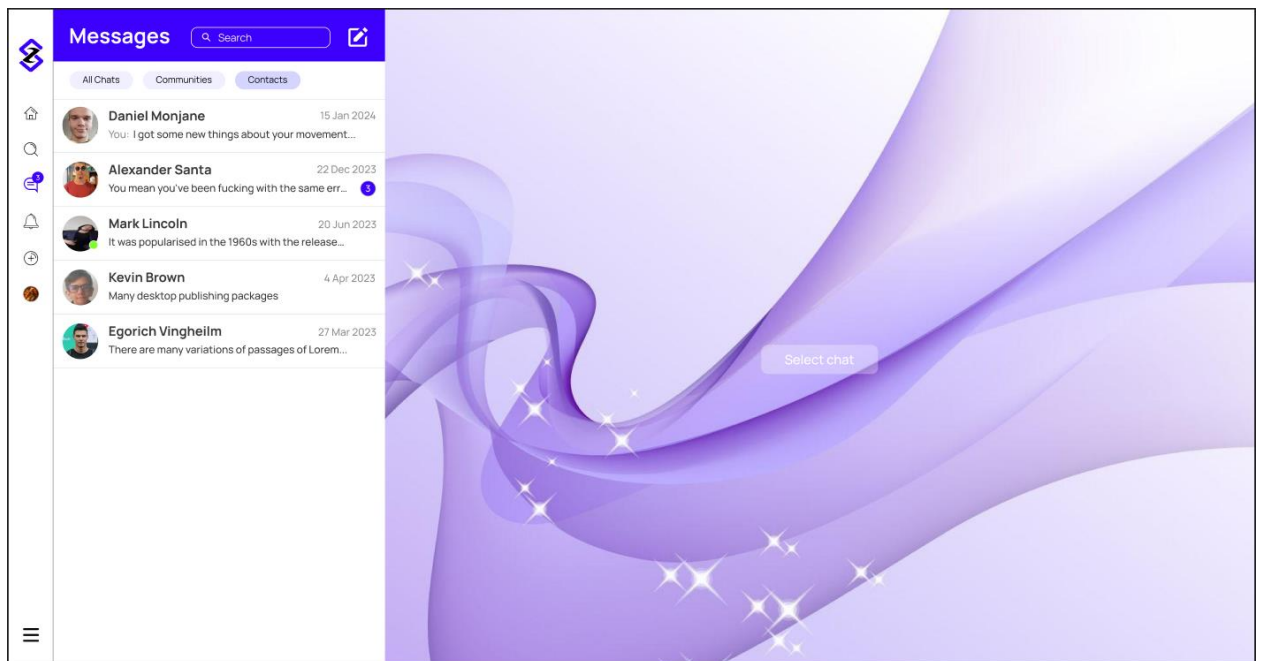
3.2.5 Поиск

Пользователь также может использовать поиск через боковое меню, в котором может ввести название/имя того, чего ищет. В результате будет выдан список из найденных сообществ и людей по определенному имени или никнейму, за один запрос одновременно можно получить результат в количестве 50 записей. Записи будут идти от самых подходящих, то есть те записи, на которые знакомые пользователя подписаны, до самых обычных похожих по введённому запросу (см Рисунок 13).

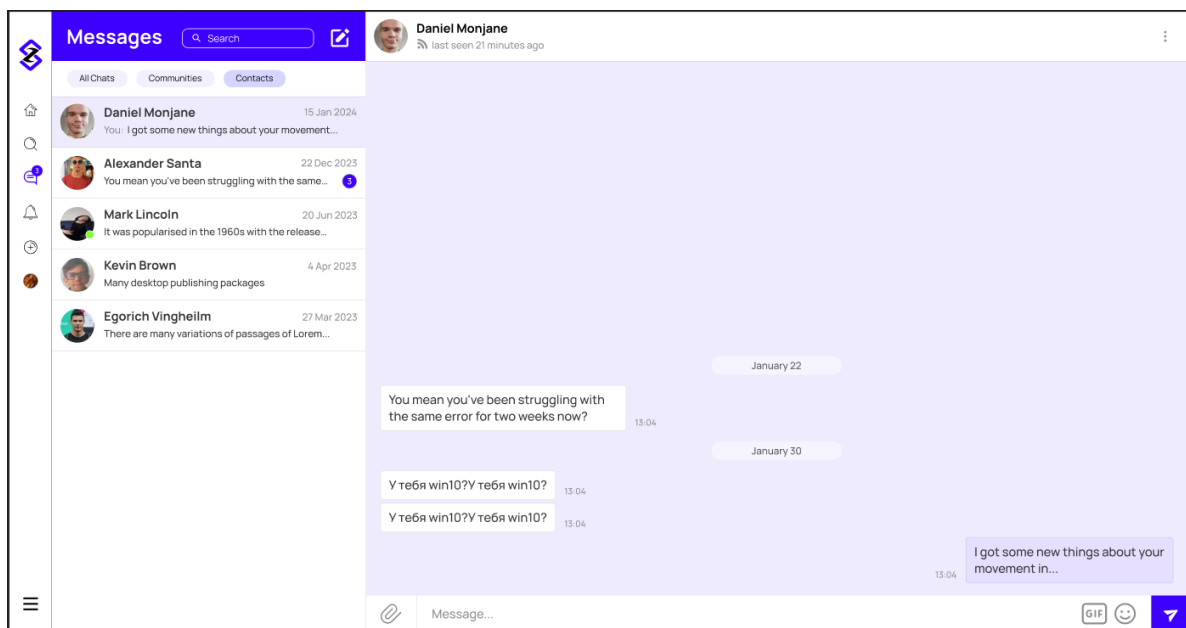


3.2.6 Мессенджер

Пользователь может перейти в мессенджер нажав на кнопку 'Messages' в боковом меню. Откроется страница со всеми чатами пользователя, там имеется фильтрация по типу чата, поскольку есть три вида чатов: Приватный, Групповой и чат с Сообществом. Каждый чат имеет информацию последнего сообщения, также если чат имеет непрочитанные сообщения пользователем, то они будут отображаться в фиолетовом кружке с количеством непрочитанных. Если чат приватный, то в случае, если собеседник данного чата в сети, будет показываться зеленый кружок рядом с изображением чата. Можно также ввести в поиске название чата, чтобы облегчить поиск определенного чата. Через поиск можно также находить чаты, в которых пользователь почистил историю сообщений, потому что чаты без сообщений не отображаются в списке. Ну и конечно, также присутствует кнопка создания чата (см Рисунок 14).



Нажав на какой-то чат из списка, появиться сам чат, с сообщениями, верхним меню и с полем для ввода. Можно также отправлять GIF и эмодзи открыв нужное меню в нижней части экрана рядом с кнопкой отправки. В верхнем меню отображается контент относительно типа чата, если чат приватный, то будет показываться краткая информация собеседника, если чат групповой, то будет показываться названия чата и количество участников, если открыт чат с сообществом, соответственно будет показываться краткая информация о сообществе, то есть имя и надпись 'Community' (см Рисунок 15).

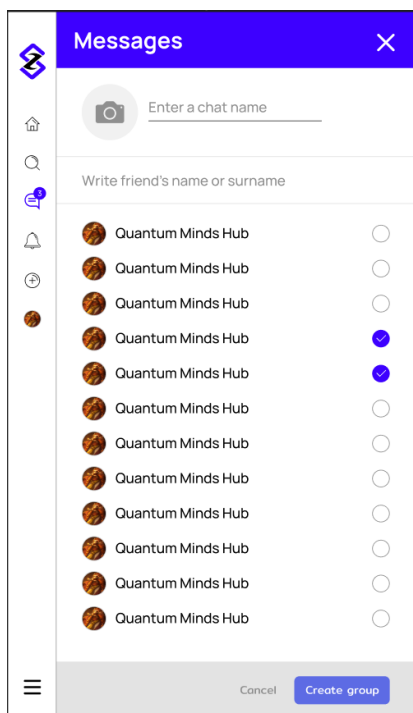


Нажав на меню в правом верхнем углу, откроется панель взаимодействия с чатом. Можно отключить уведомления из чата и в любой момент можно включить их обратно. Также

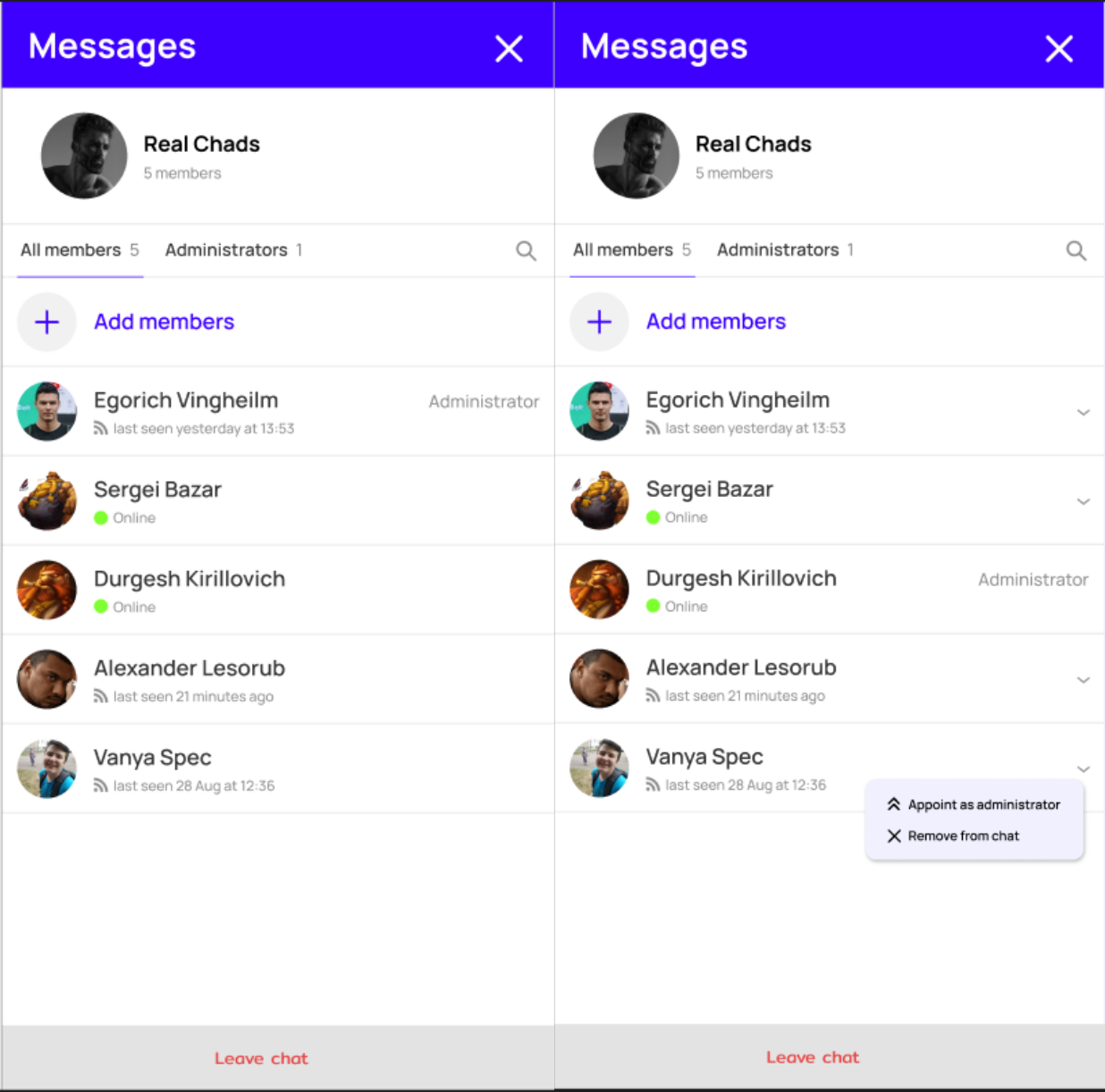
можно очистить историю сообщений, при этом чат будет скрыт в списке чатов до того момента, пока в него кто-то не напишет сообщение. Если же чат групповой, то есть возможность выйти из него, пользователь не будет получать ни уведомлений, ни сообщений из этого чата, но можно также вернуться в него, если пользователь вышел по своей инициативе, а не из-за того, что администрация чата выгнала данного пользователя (см Рисунок 16).



Нажав на кнопку карандаша в левом верхнем фиолетовом меню на странице чатов, можно перейти к созданию нового чата. Пользователь может загрузить изображение, задать название и добавить своих друзей в эту беседу. В зависимости от действий пользователя можно создать два типа чатов. Если пользователь задал имя, загрузил изображение или добавил двух или более друзей, то чат будет групповым. В случае, если добавлен один участник и не задано ни одно поле, то откроется приватный чат с выбранным собеседником (см Рисунок 17).



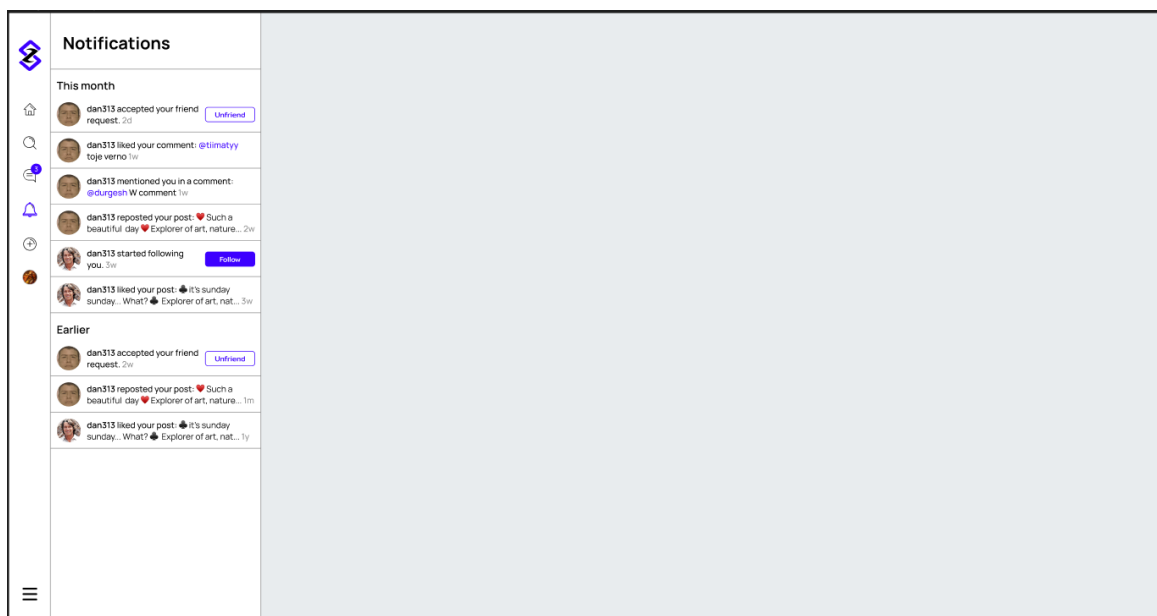
В случае, если открыт групповой чат, можно нажать по краткой информации чата и откроется меню с информацией о группе. Название, изображение, количество участников, кнопка добавления новых участников и кнопка выхода из чата. Вид участников зависит от того, какую роль имеет участник в данном чате, если это администратор или создатель, то также будут видны стрелочки, на которых при наведении появляется меню с взаимодействием с участником. Создатель может обозначить участника администратором и также его понизить до роли обычного участника, также есть возможность выпнуть участника из группы (см Рисунок 18).



3.2.7 Уведомления

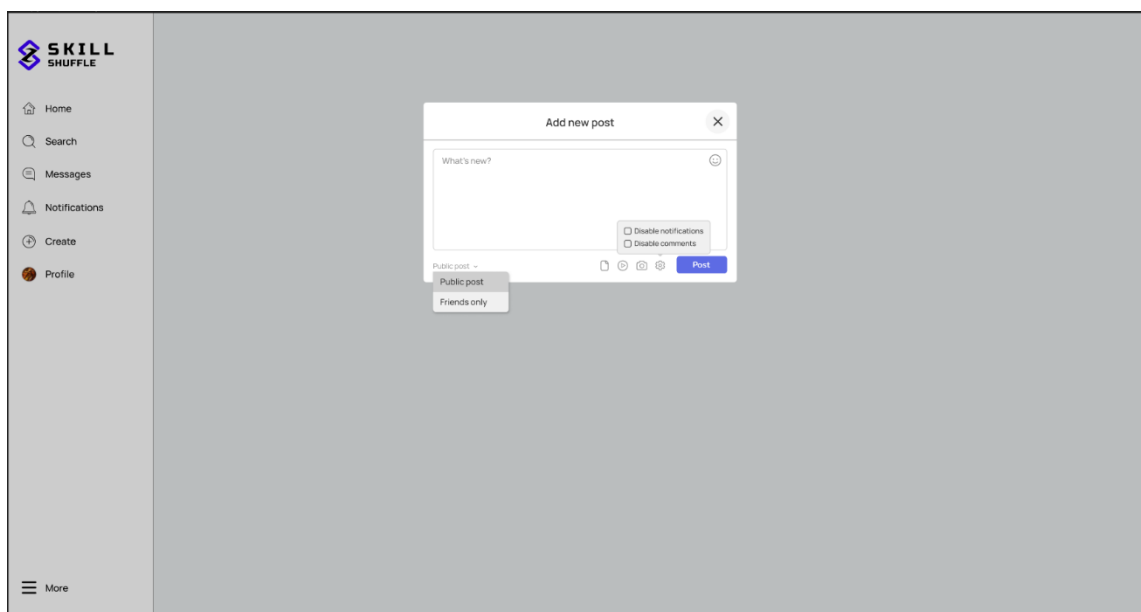
Через боковое меню можно посмотреть уведомления нажав на иконку с колоколом. При нажатии выведутся все полученные уведомления в порядке убывания по времени получения (см Рисунок 19). За уведомление считаются если кто-то:

- поставил отметку нравится вашей публикации, комментарию
- подписался на пользователя
- выделил пользователя в публикации, комментарии
- принял заявку в друзья
- поделился публикацией пользователя на своей странице
- добавил новую публикацию



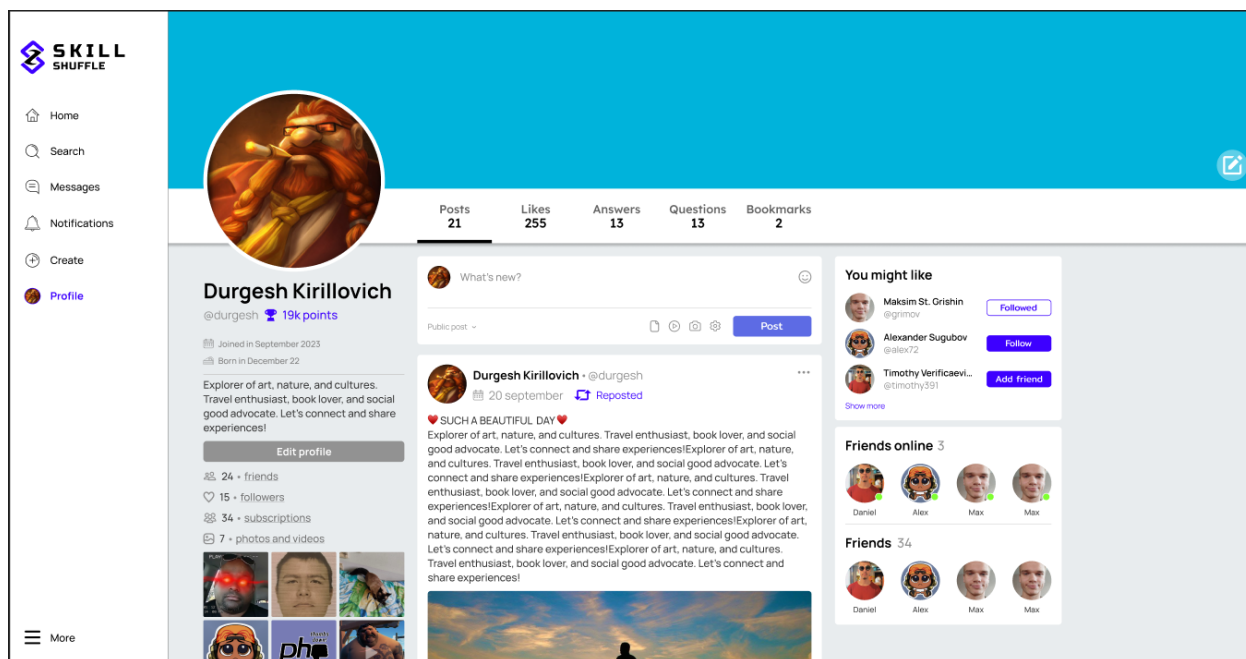
3.2.8 Создание публикации

Нажав на кнопку ‘Create’ в боковом меню, откроется модальное окно с инструментами создания записи. Можно написать текст, использовать эмодзи, также добавлять видео, фото и файлы. Присутствует также настройка приватности публикации, можно поставить видимость для всех пользователей или только для друзей. Есть возможность отключить комментарии к публикации и отключить рассылку уведомлений подписанным на пользователя людям, у которых включены уведомления о новых публикациях пользователя (см Рисунок 20).



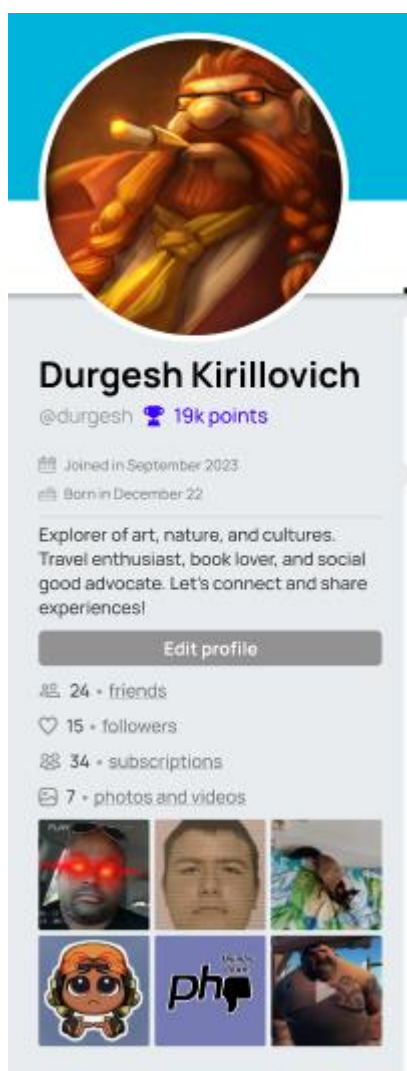
3.2.9 Страница профиля

Самой неотъемлемой частью социальных сетей является наличие страницы с информацией о пользователе, её можно открыть с помощью бокового меню, нажав на изображение профиля с текстом 'Profile' (см Рисунок 21).



Открыв страницу профиля, в левой части экрана можно увидеть следующие элементы с информацией о пользователе (см Рисунок 22):

- баннер профиля
- изображение (аватарка) пользователя
- никнейм пользователя
- количество баллов на сайте
- дата регистрации
- дата рождения
- биография
- кнопка редактирования профиля
- количество друзей
- количество подписчиков
- количество подписок на сообщества
- фотографии и видео пользователя

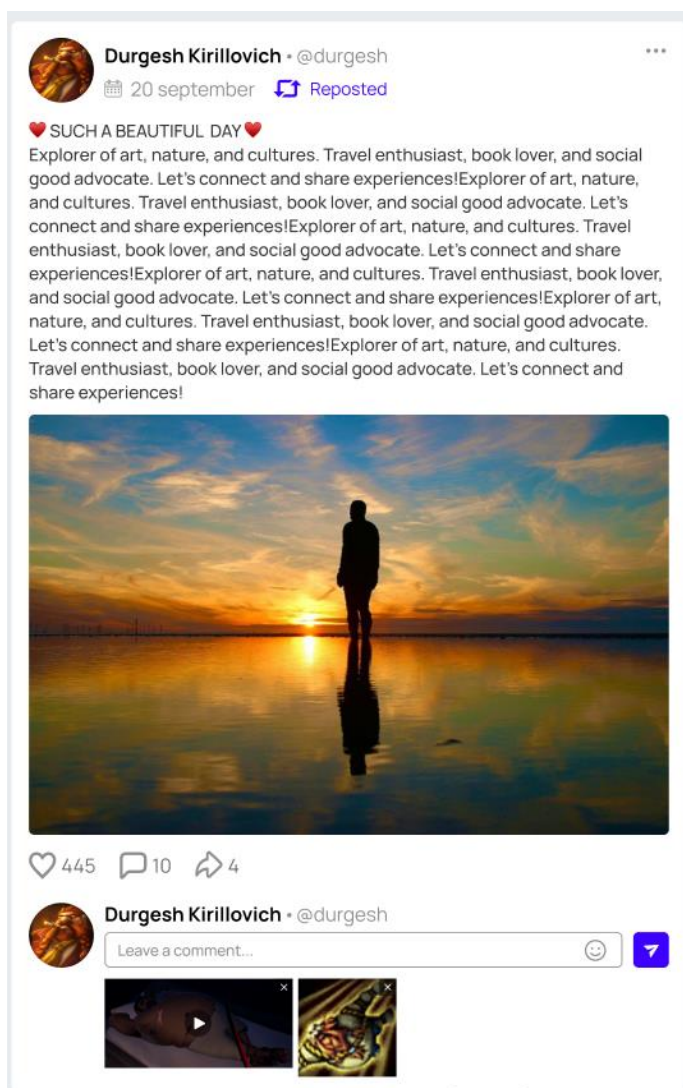


Сверху также имеется навигационное поле, с помощью которого можно переключать тип отображаемого контента. По умолчанию выбраны публикации пользователя, можно также выбрать понравившиеся публикации пользователем, ответы на заданные другими пользователями вопросы, вопросы от пользователя, закладки (сохраненные публикации) (см Рисунок 23).



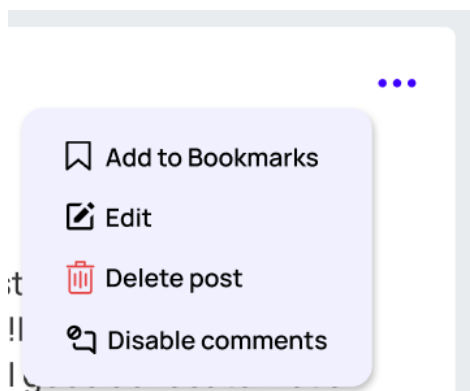
Публикации, отображаемые на стене профиля, имеют краткую информацию о создателе публикации, дату создания, пометка 'Reposted', если пользователь поделился данной публикацией от другого человека или сообщества. Публикация также включает в себя количество лайков, комментариев и «репостов» (поделившиеся данной публикацией пользователи). Также есть поле создания комментария к публикации, если комментарии не

отключены. В комментарии можно писать текст, эмодзи и отправлять файлы, фото и видео, всего можно вложить два файла в один комментарий (см Рисунок 24).



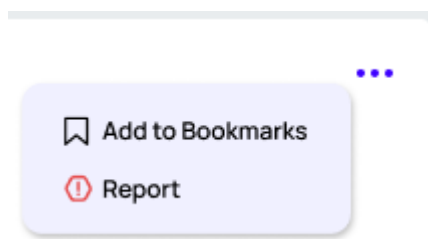
Также публикация имеет меню взаимодействия в верхнем правом углу. Нажав на него, откроется окно с действиями, отображаемые действия зависят от того, являетесь ли вы создателем публикации. Если вы являетесь автором публикации, то будут отображены следующие действия (см Рисунок 25):

- Редактировать
- Удалить
- Добавить в закладки
- Отключить комментарии

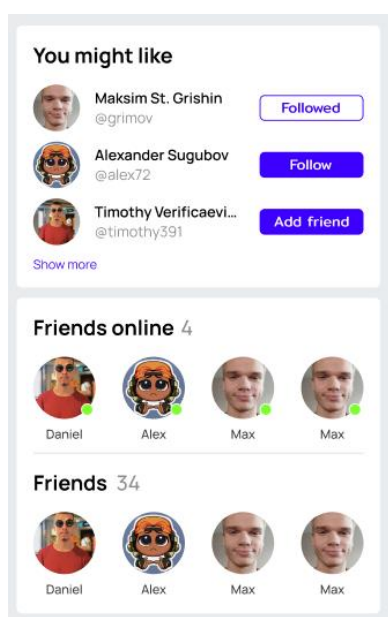


Если же пользователь не является автором публикации, отображено будет только (см Рисунок 26):

- Добавить в закладки
- Пожаловаться

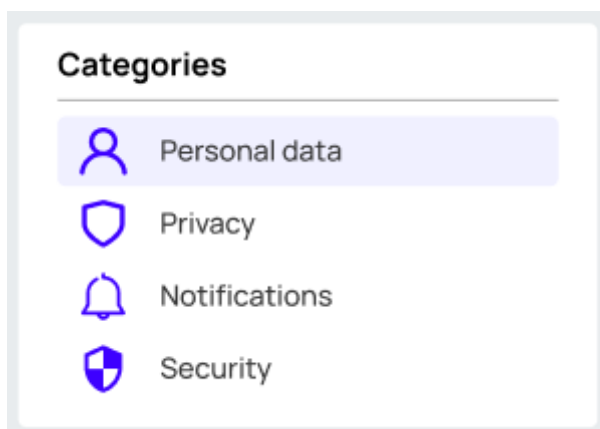


В правой части страницы профиля отображены окна «Вы можете знать» и «Друзья». Через окно «Вы можете знать» можно легко находить возможных пользователю знакомых, которые дружат с друзьями пользователя. А через панель с друзьями можно отслеживать какие друзья в данный момент в сети и краткий список друзей пользователя. Нажав на заголовок, можно открыть полный список (см Рисунок 27).



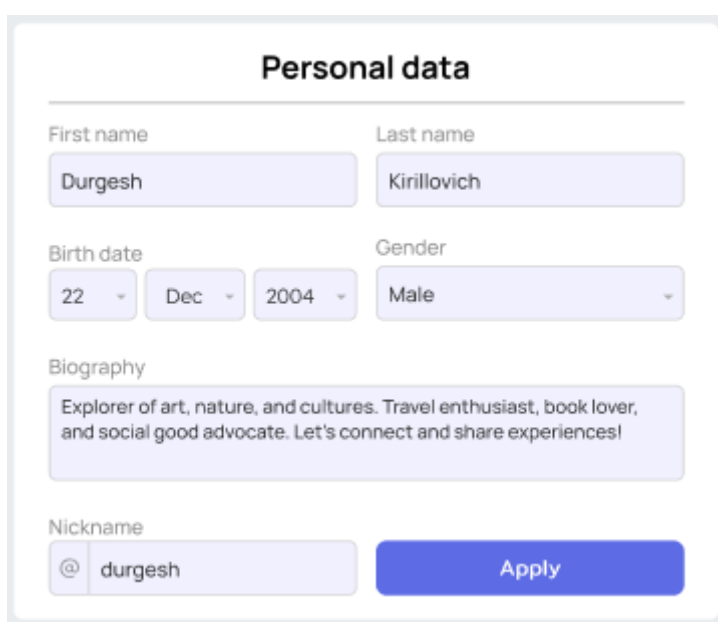
3.2.10 Редактирование профиля

Если пользователь нажмет кнопку 'Edit profile', то откроется страница редактирования профиля. По умолчанию отображается окно личных данных, но можно также выбрать другие категории, например конфиденциальность, уведомления и безопасность (см. Рисунок 28).

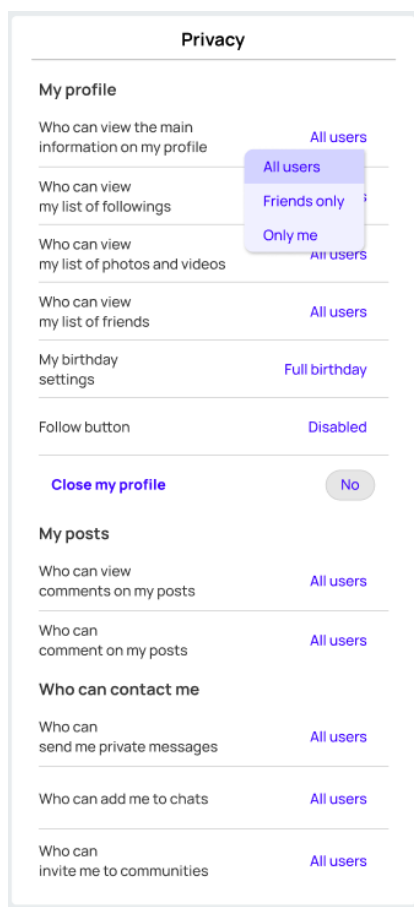


Выбрав категорию «личные данные», можно поменять такие поля как (см. Рисунок 29):

- Имя
- Фамилия
- Дата рождения
- Пол
- Биография
- Никнейм

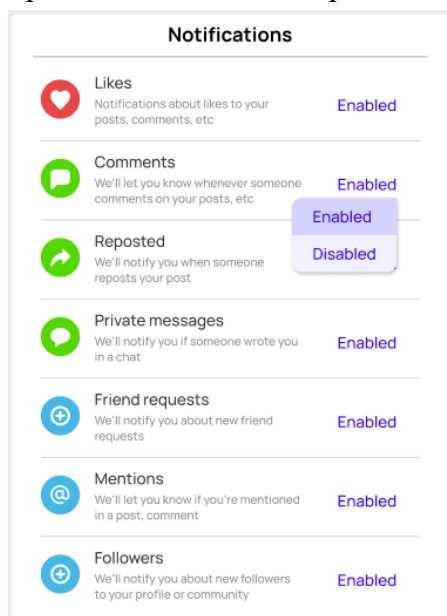
The image shows the 'Personal data' form. It includes fields for 'First name' (Durgesh) and 'Last name' (Kirillovich). Below these are 'Birth date' (22 Dec 2004) and 'Gender' (Male). There is a 'Biography' text area with the text: 'Explorer of art, nature, and cultures. Travel enthusiast, book lover, and social good advocate. Let's connect and share experiences!'. At the bottom, there is a 'Nickname' field with '@ durgesh' and an 'Apply' button.

Выбрав категорию «конфиденциальность», пользователь может выбрать политику отображения данных, связанных с профилем. Например, можно ограничить круг лиц, которые могут посмотреть список друзей или публикации пользователя (см. Рисунок 30).



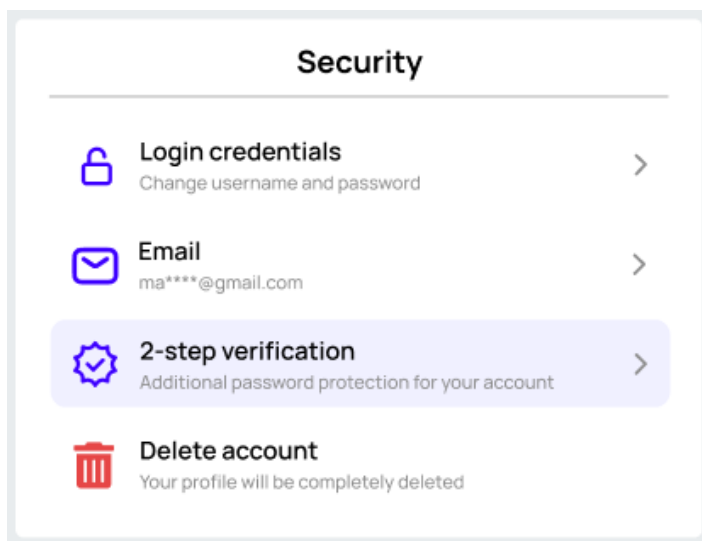
Privacy	
My profile	
Who can view the main information on my profile	All users
Who can view my list of followings	All users
Who can view my list of photos and videos	All users
Who can view my list of friends	All users
My birthday settings	Full birthday
Follow button	Disabled
Close my profile	No
My posts	
Who can view comments on my posts	All users
Who can comment on my posts	All users
Who can contact me	
Who can send me private messages	All users
Who can add me to chats	All users
Who can invite me to communities	All users

Категория «уведомления» включает в себя список всех существующих типов уведомлений и их статус. Пользователь может отключить или включить уведомления об отметках нравится или комментариев под публикацией и так далее (см. рисунок 31).



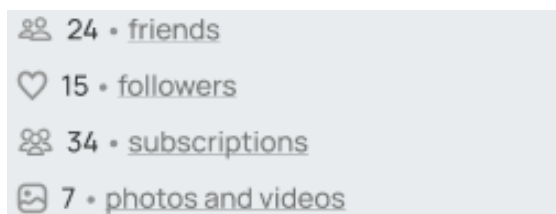
Notifications	
Likes Notifications about likes to your posts, comments, etc	Enabled
Comments We'll let you know whenever someone comments on your posts, etc	Enabled
Reposted We'll notify you when someone reposts your post	Disabled
Private messages We'll notify you if someone wrote you in a chat	Enabled
Friend requests We'll notify you about new friend requests	Enabled
Mentions We'll let you know if you're mentioned in a post, comment	Enabled
Followers We'll notify you about new followers to your profile or community	Enabled

Выбрав категорию «Безопасность», пользователь может удалить аккаунт, поменять почту, добавить двухфакторную аутентификацию с помощью которого будет присылаться код подтверждения на почту, ну и конечно же пользователь может поменять данные для входа (см. Рисунок 32).

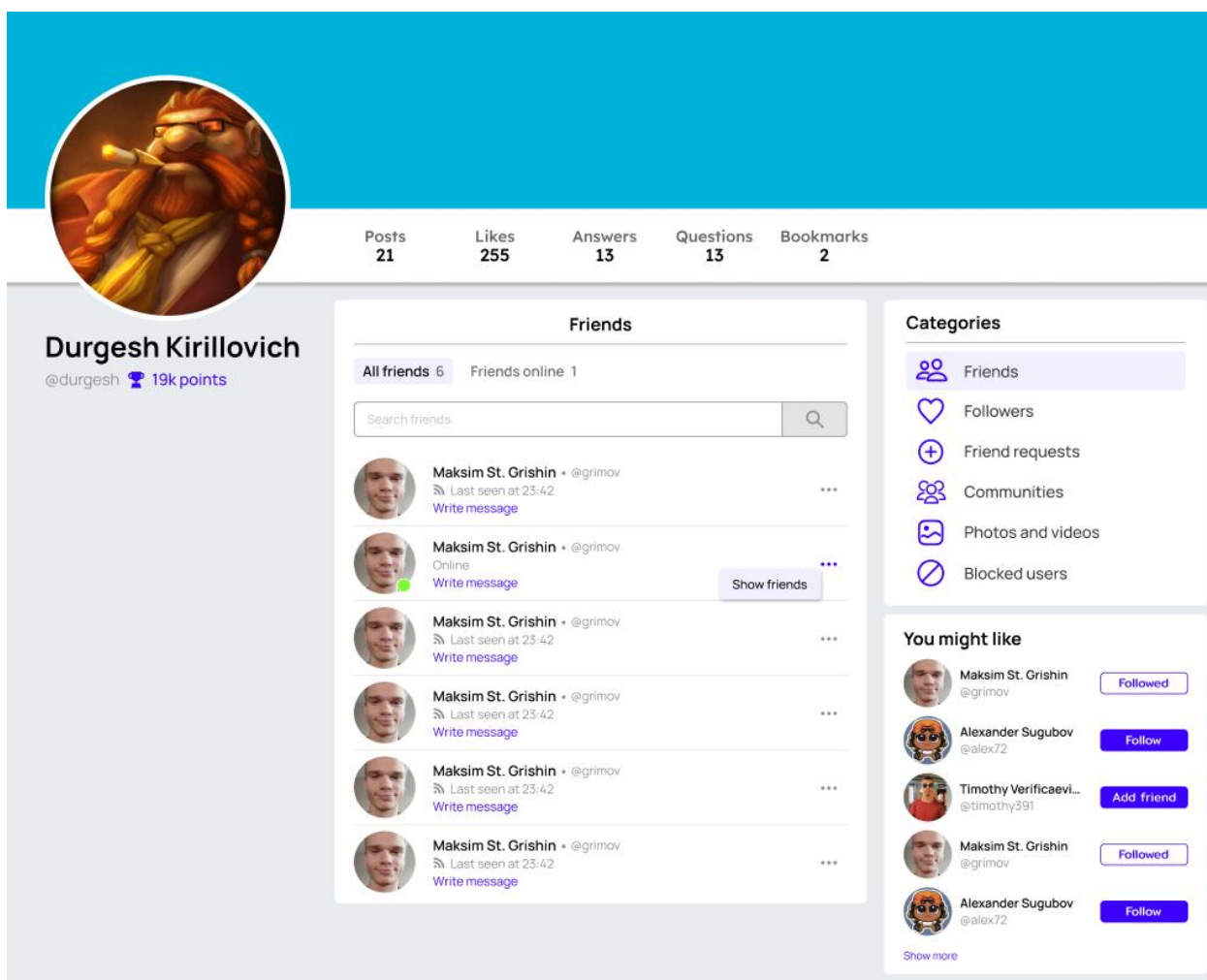


3.2.11 Интерактивные элементы профиля

Через страницу профиля можно также открыть список друзей, подписчиков, подписок, сообществ, заявок в друзья, фото и видео, заблокированных пользователей (см. Рисунок 33).

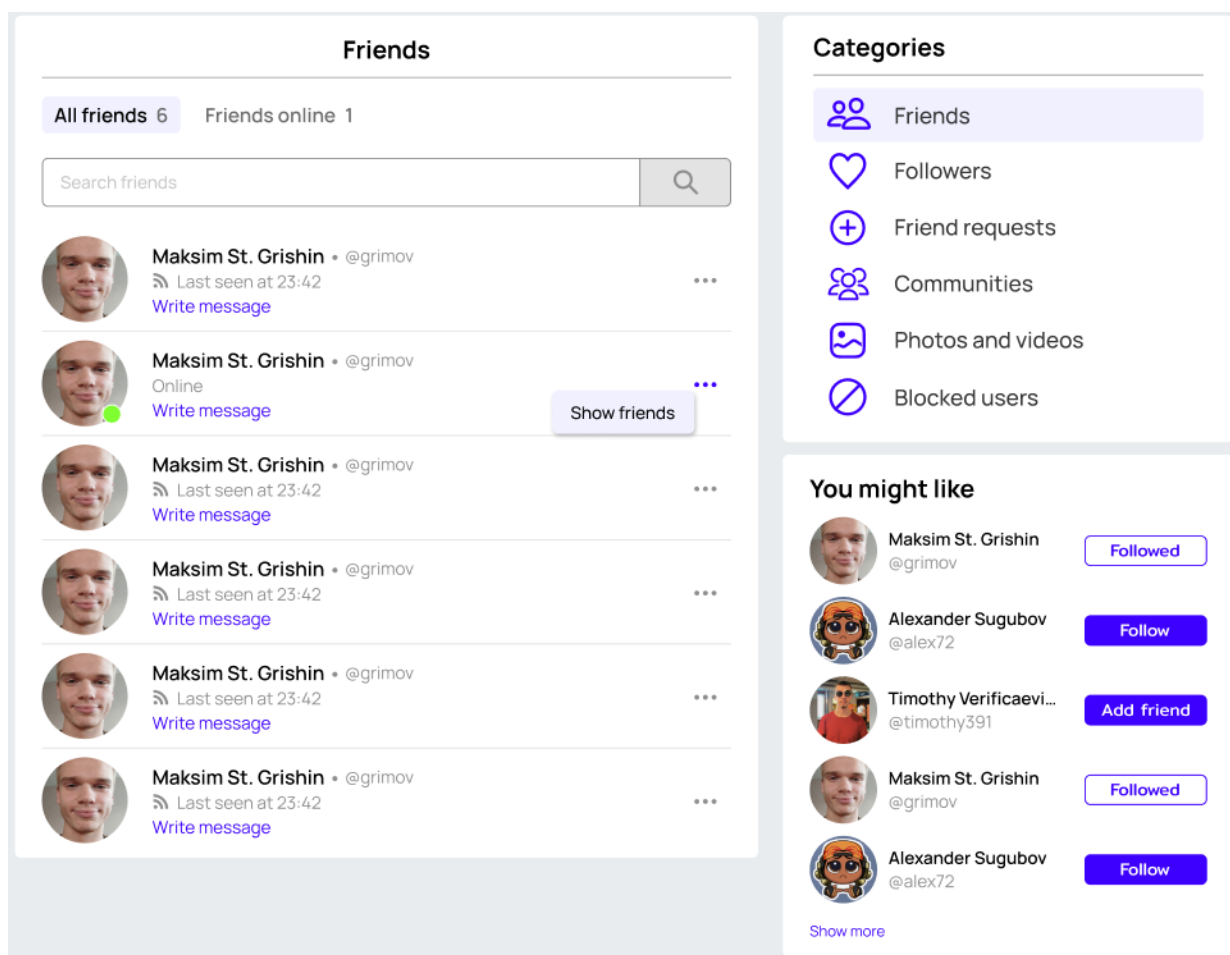


Нажав на что-либо из изображения (см. Рисунок 33), откроется страница с выбранной категорией (см. Рисунок 34).



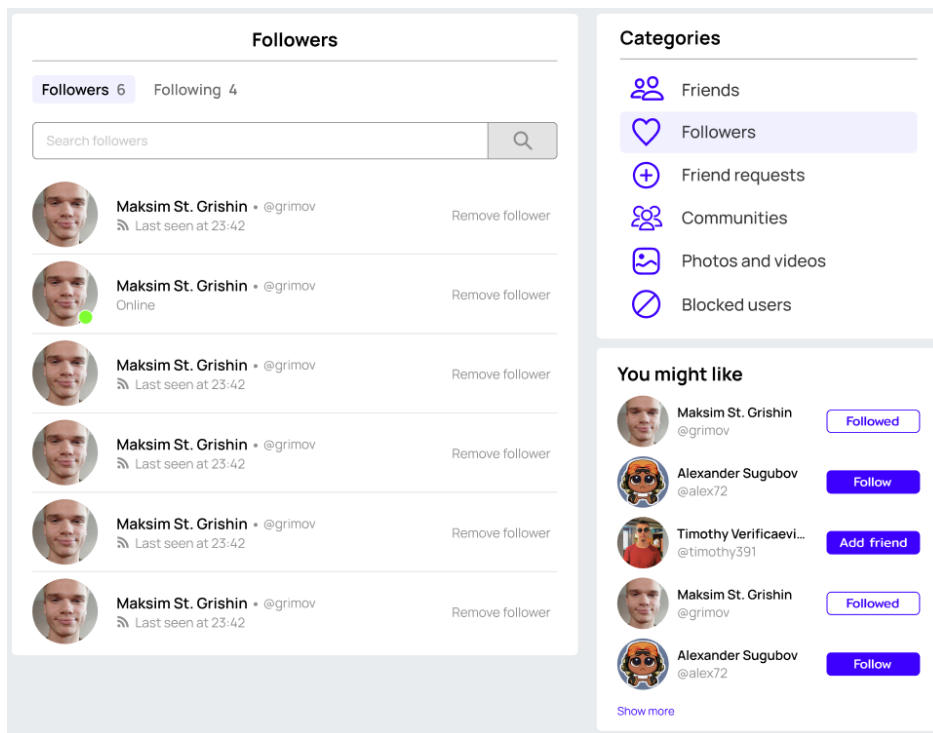
3.2.12 Список друзей

Нажав на кнопку «друзья» появится список друзей, который можно отфильтровать по друзьям в онлайн и всем друзьям. Каждый отображенный друг имеет краткую информацию о себе, кнопку «написать сообщение» и меню, наведясь на которое, всплывет окно с кнопкой «показать друзей» (см. Рисунок 35).



3.2.13 Подписчики

Выбрав категорию «подписчики» откроется панель со списком подписчиков, подписчиков можно удалять из списка. Имеется также возможность посмотреть список людей на которых пользователь подписан, это можно сделать, нажав на кнопку «подписки» в фильтрации сверху. Есть также возможность отписаться от пользователя просто нажав на кнопку «отписаться» рядом с пользователем (см. Рисунок 36).



3.2.14 Заявки в друзья

Нажав на категорию «заявки в друзья», откроется панель со списком заявок. Каждая заявка имеет краткую информацию о человеке и две кнопки. Одна кнопка отвечает за игнорирование заявки, для того чтобы она исчезла из списка заявок, а нажав на другую кнопку пользователь принимает эту заявку, и пользователи становятся друзьями (см. Рисунок 37). Можно также посмотреть заявки, отправленные пользователем нажав на кнопку 'Outgoing' в верхней фильтрации. Там такой же вид заявок, только кнопка всего одна, в зависимости от действия, это может быть кнопка «Отписаться» или «Отменить запрос» (см. Рисунок 38).

Friend requests

Pending 6

Outgoing 1



Maksim St. Grishin • @grimov

Last seen at 23:42

Accept request

Ignore



Maksim St. Grishin • @grimov

Online

Accept request

Ignore



Maksim St. Grishin • @grimov

Last seen at 23:42

Accept request

Ignore



Maksim St. Grishin • @grimov

Last seen at 23:42

Accept request

Ignore



Maksim St. Grishin • @grimov

Last seen at 23:42

Accept request

Ignore



Maksim St. Grishin • @grimov

Last seen at 23:42

Accept request

Ignore

Categories

 Friends

 Followers

 Friend requests

 Communities

 Photos and videos

 Blocked users

You might like



Maksim St. Grishin

@grimov

Followed



Alexander Sugubov

@alex72

Follow



Timothy Verificaevi...

@timothy391

Add friend



Maksim St. Grishin

@grimov

Followed



Alexander Sugubov

@alex72

Follow

Show more

Friend requests

Pending 6

Outgoing 2



Maksim St. Grishin • @grimov

Last seen at 23:42

Alexander Sugubov and 3 other mutual friends

Cancel request



Maksim St. Grishin • @grimov

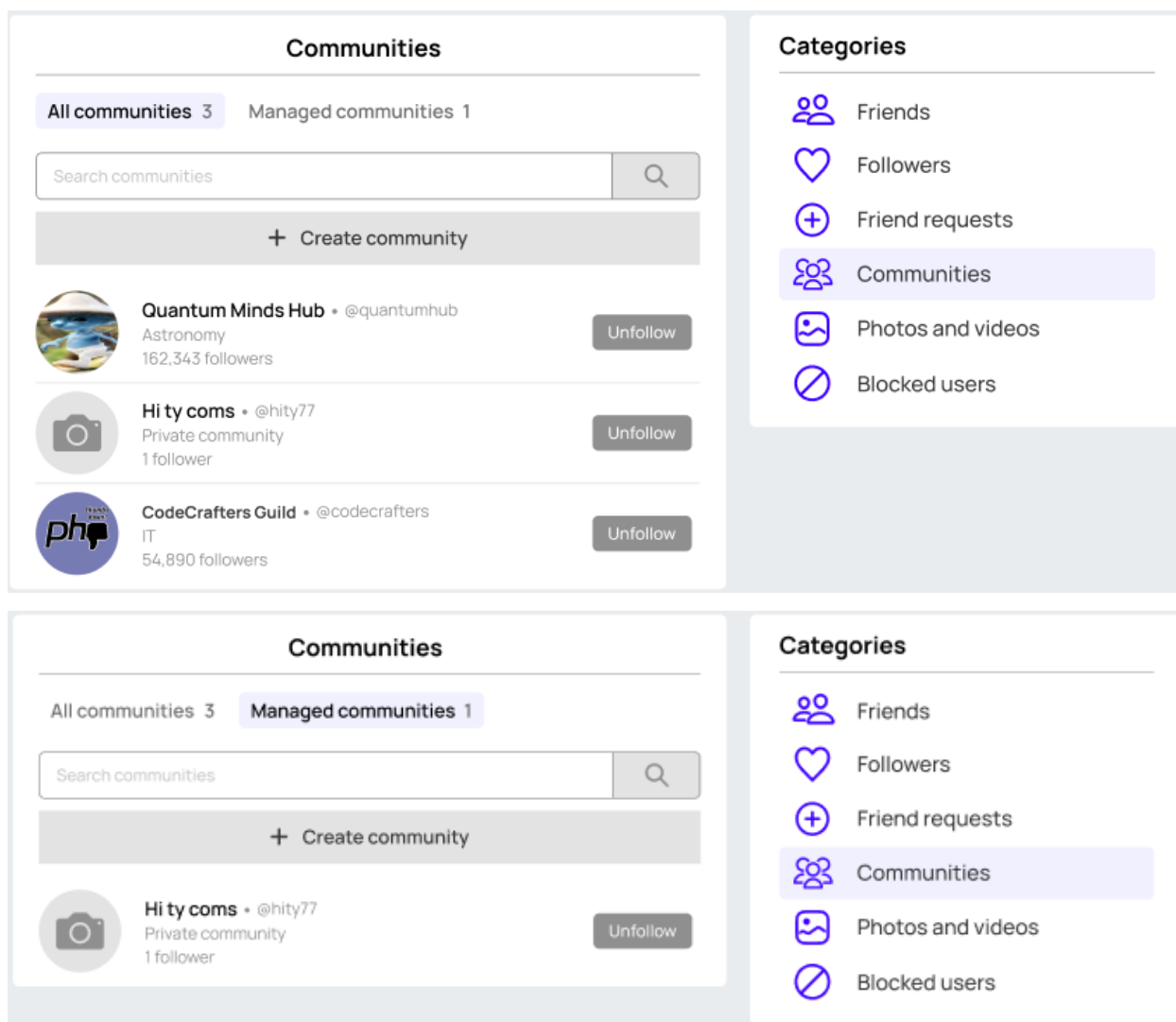
Online

Unfollow

38

3.2.15 Список сообществ

Можно также посмотреть список сообществ, на которые пользователь подписан. Для этого нужно нажать на категорию «сообщества», нажав на категорию, появится панель со списком сообществ (см. Рисунок 39). Каждое сообщество имеет краткую информацию о сообществе и кнопку «отписаться». Если пользователь хочет посмотреть сообщества, которыми он владеет (см. Рисунок 40), можно нажать на кнопку «Managed communities» в панели фильтрации. Имеется также кнопка создания сообщества.



3.2.16 Создание сообщества

Нажав на кнопку создания сообщества, всплывет модальное окно с пошаговым созданием (см. Рисунок 41). Пользователя попросят написать название сообществу, никнейм и тип публичности (открытое, приватное, закрытое), тему сообщества. Это обязательные шаги для создания сообщества, можно продолжить создание нажав на кнопку «Продолжить настройку» в окне, которое появится после второго шага.

The image displays two side-by-side screenshots of a 'Create community' modal window, illustrating the two-step process.

Left Screenshot (Step 1 of 2):

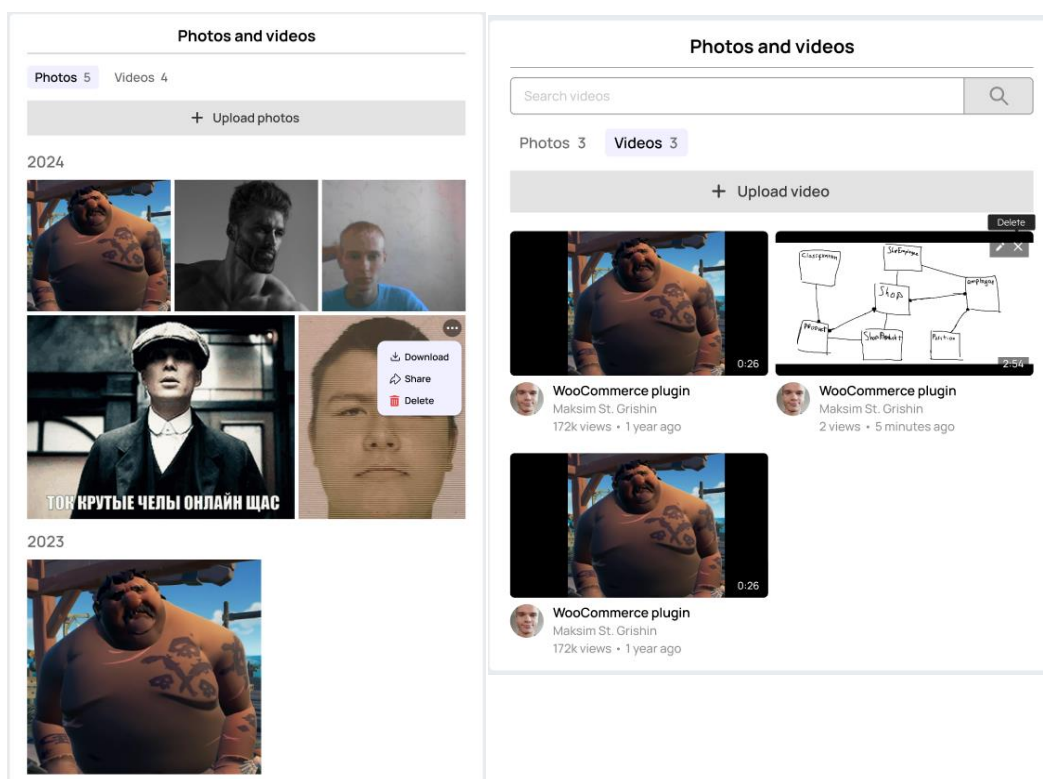
- Title:** 'Come up with a title'. Subtext: 'Use words that reflect the idea of the community. You can change the name later.'
- Community name:** Input field with placeholder 'Enter a name' and character count '0/48'.
- Community nickname:** Input field with placeholder '@ Enter a nickname' and character count '0/30'.
- Community privacy:** A list of three options: 'Open' (selected with a blue checkmark), 'Closed', and 'Private'. Each option has a brief description of its visibility settings.
- Buttons:** 'Cancel' and 'Continue'.

Right Screenshot (Step 2 of 2):

- Title:** 'Select subject'. Subtext: 'Your selection won't affect the community possibilities. You can edit the community subject at any time.'
- Search:** A search bar with placeholder 'Search subject'.
- Subject List:** A vertical list of subjects: 'Computer science', 'Psychology', 'Physics', 'Biology', 'Chemistry' (highlighted), 'Mathematics', and 'Philosophy'.
- Buttons:** 'Back' and 'Create community'.

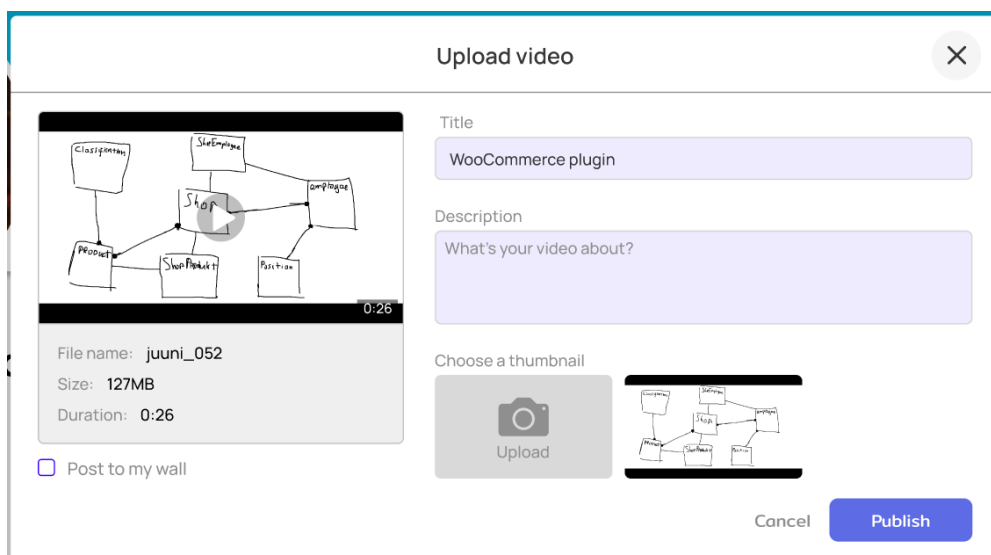
3.2.17 Фото и видео профиля

Нажав на категорию «фото и видео» откроется панель с фотографиями пользователя (см. Рисунок 42), через панель фильтрации можно просмотреть видео пользователя.



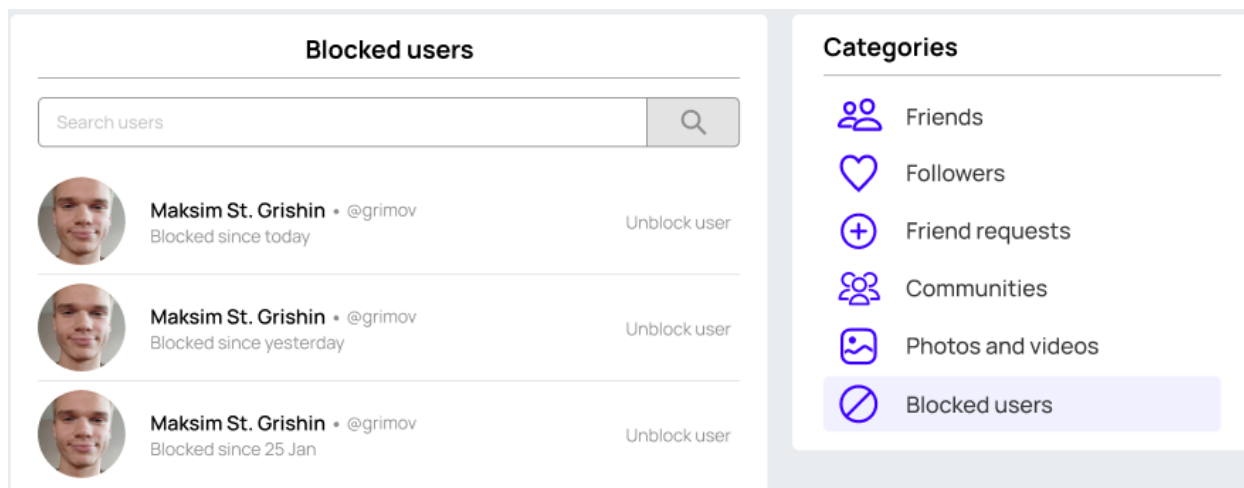
3.2.18 Загрузка видео

Пользователь может загрузить новые видео или фото, нажав на кнопку загрузки, она есть во вкладках видео и фото. Нажав на кнопку «загрузить видео», откроется модальное окно с добавлением видео (см. Рисунок 43).



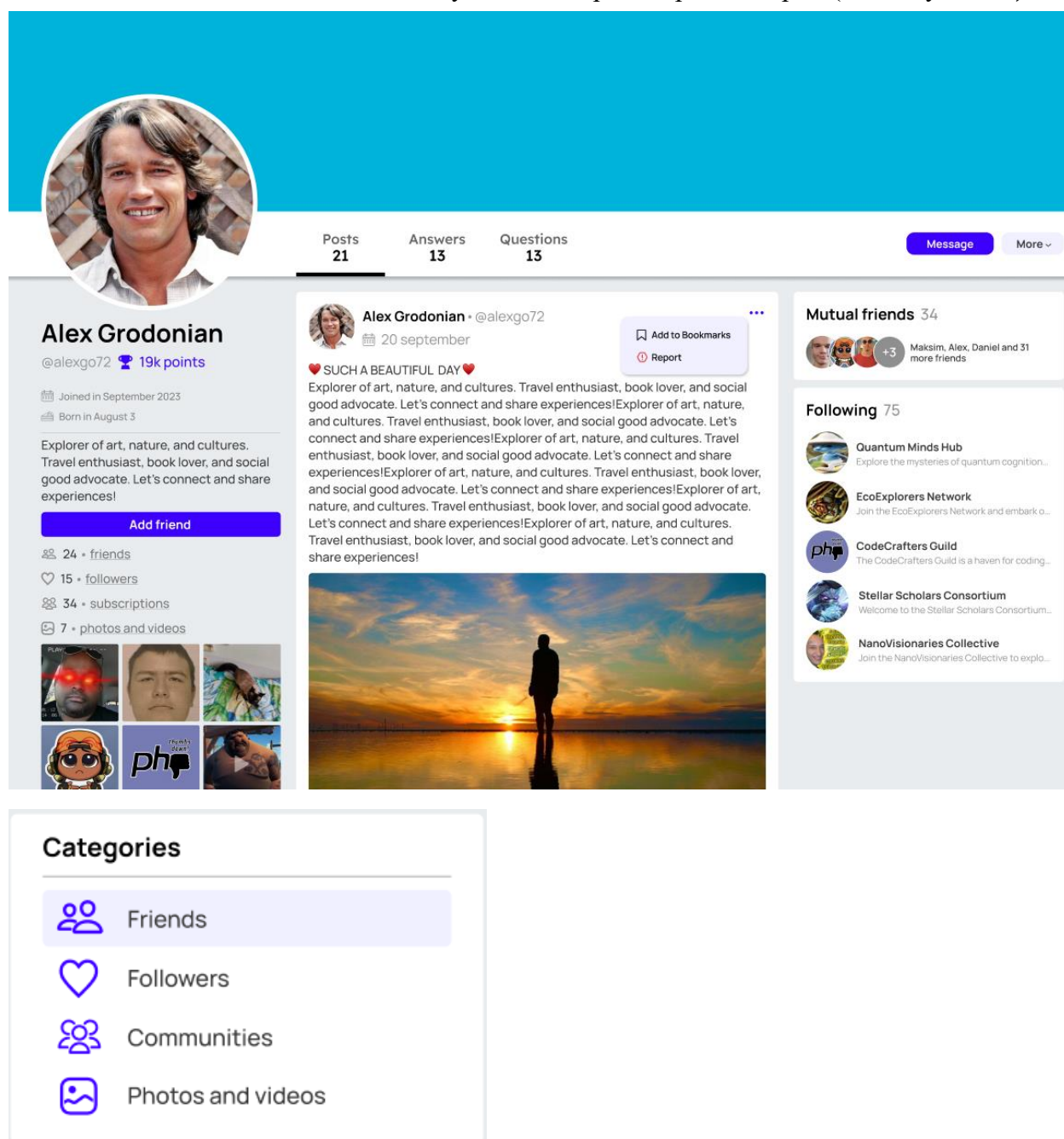
3.2.19 Список заблокированных пользователей

Нажав на кнопку «заблокированные пользователи», откроется список с заблокированными пользователями, каждый пользователь имеет имя, никнейм, дату блокировки и кнопку «разблокировать пользователя» (см. Рисунок 44).



3.2.20 Страница пользователя

Перейдя на аккаунт другого пользователя, открывается страница, практически полностью схожая со страницей профиля, отличаются всего несколько элементов (см. Рисунок 45). Например, кнопка действия, добавились кнопки «написать» и «ещё» в верхнем меню, боковое меню с общими друзьями и списком подписанных сообществ. Через кнопку «ещё» можно отключить/включить уведомления о новых публикациях пользователя, также пользователя можно заблокировать/разблокировать и можно отправить жалобу на пользователя. Также отличаются доступные для просмотра категории (см. Рисунок 46)

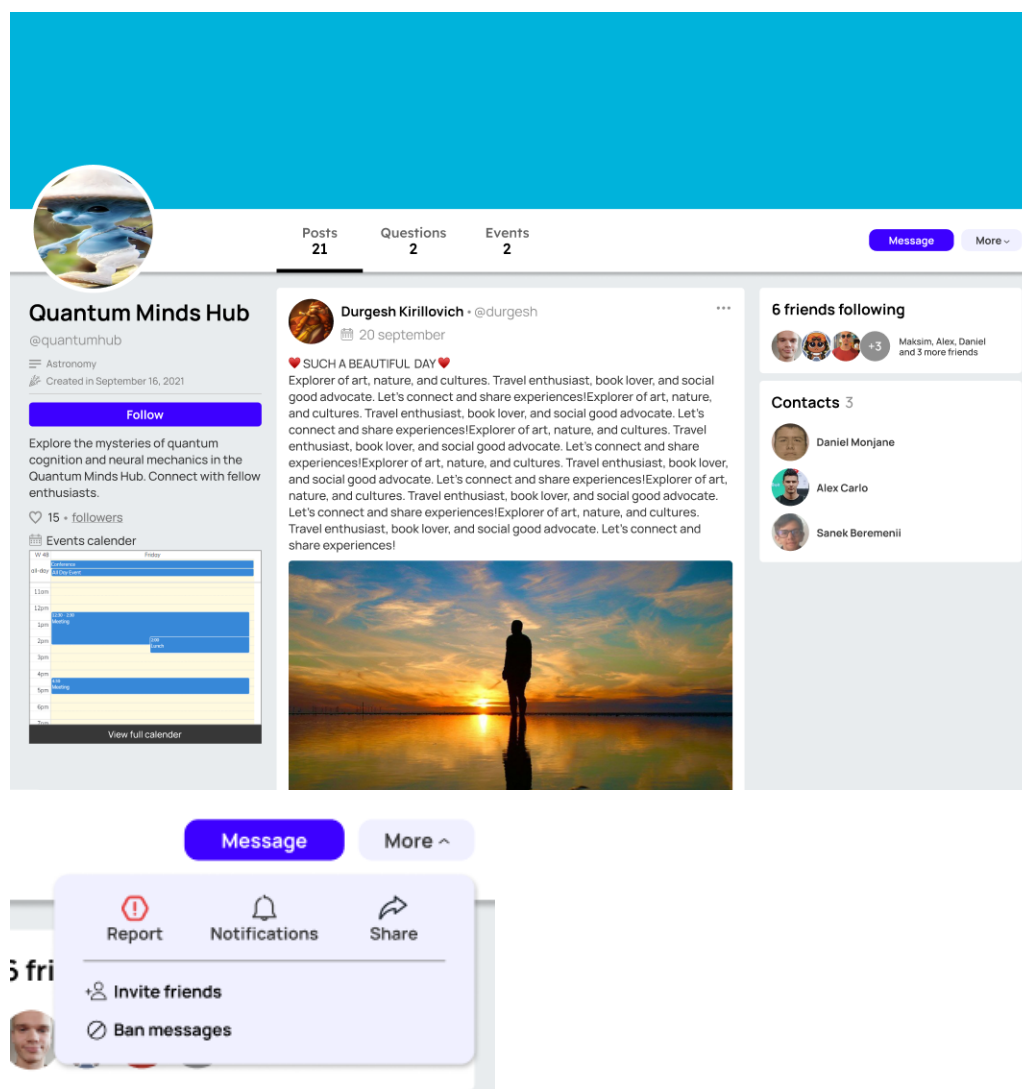


3.2.21 Страница сообщества

Перейдя на страницу любого сообщества, следующая страница (см. Рисунок 47). Она также похожа на страницу профиля, но есть несколько нюансов. Теперь в левой части страницы показывается:

- тематика сообщества
- дата создания
- кнопка подписаться/отписаться
- описание
- количество подписчиков
- календарь событий

В правой панели отображаются главные лица сообщества и друзья, которые подписаны на данное сообщество. Также кнопка «ещё» в верхнем меню имеет новые поля (см. Рисунок 48).



Также в выборе контента в верхнем меню добавилась вкладка событий, нажав на неё будут выводиться события в этом сообществе (см. Рисунок 49).

Posts21

Questions2

Events2



Quantum Minds Hub • by Durgesh Kirillovich

Today at 16:35 WEBINAR

Physics Unveiled: A Deep Dive into the Mysteries of the Universe

🕒 26 Aug 15:00 — 30 Aug 18:00

Quantum MechanicsGravitational Waves

Join our captivating webinar dedicated to the fascinating aspects of physics! On this day, we will delve into the depths of physical laws and phenomena, unraveling the secrets of the universe. Our experts will share the latest discoveries, conduct engaging demonstrations, and answer questions, helping you understand the principles of physics on a whole new level.

JoinMark as interested



Quantum Minds Hub • by Durgesh Kirillovich

20 Aug COMPETITIVE

Physics Quest: Unleash the Challenger Within

🕒 22 Aug 12:00 — 25 Aug 20:00

Quantum QuandariesAstro Adventure

Get ready for an electrifying physics competition that will put your knowledge to the test! Physics Quest is an exhilarating challenge where participants will engage in mind-bending problems, experimental simulations, and collaborative projects. Whether you're a seasoned physicist or an aspiring enthusiast, this competition promises an adrenaline-packed journey through the thrilling realms of physics.

JoinNot Interested

Нажав на календарь событий, пользователь перейдет на страницу с календарем, в котором можно фильтровать события по дням, неделям и месяцам (см. Рисунок 50). Для разработки календаря была использована JavaScript библиотека “FullCalendar“ [10].



Quantum Minds Hub

@quantumhub

Astronomy

Created in September 16, 2021

Follow

Explore the mysteries of quantum cognition and neural mechanics in the Quantum Minds Hub. Connect with fellow enthusiasts.

15 • followers

Events calendar

View full calendar

Posts21

Questions2

Events2

December 2023

month week day list

Sun	Mon	Tue	Wed	Thu	Fri	Sat
26	27	28	29	30	31	1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29
30	31	1	2	3	4	5

6 friends following

Maksim, Alex, Daniel and 3 more friends

Contacts 3

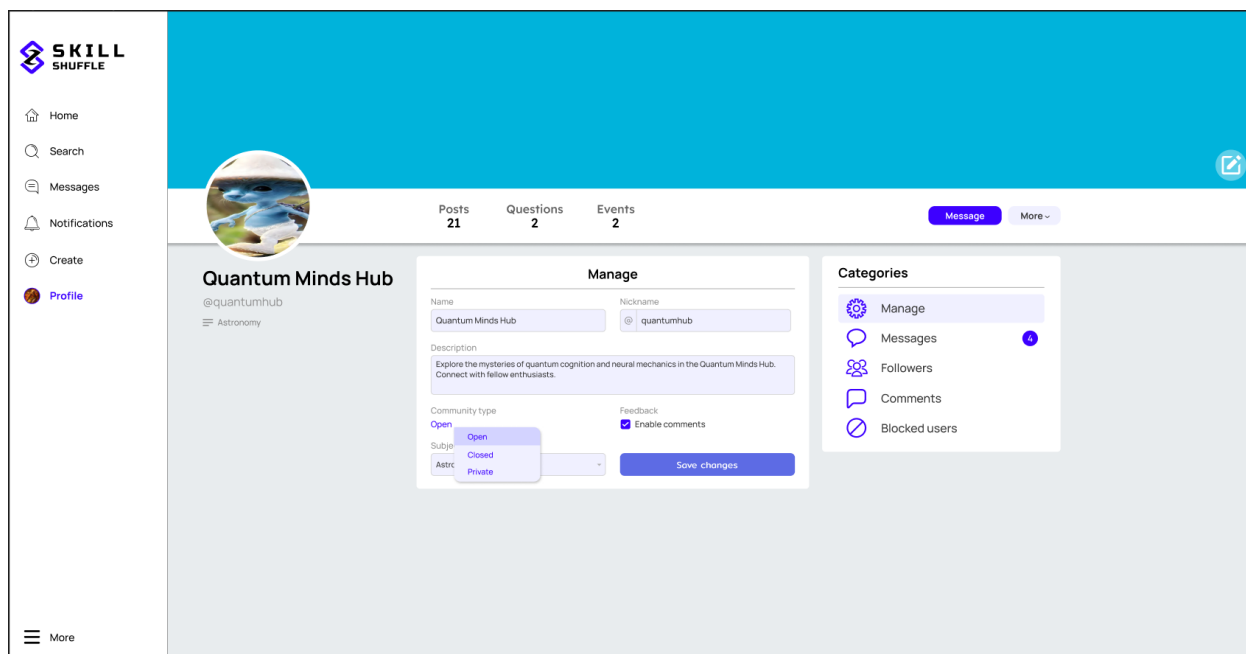
Daniel Monjane

Alex Carlo

Sanek Beremenli

3.2.22 Управление сообществом

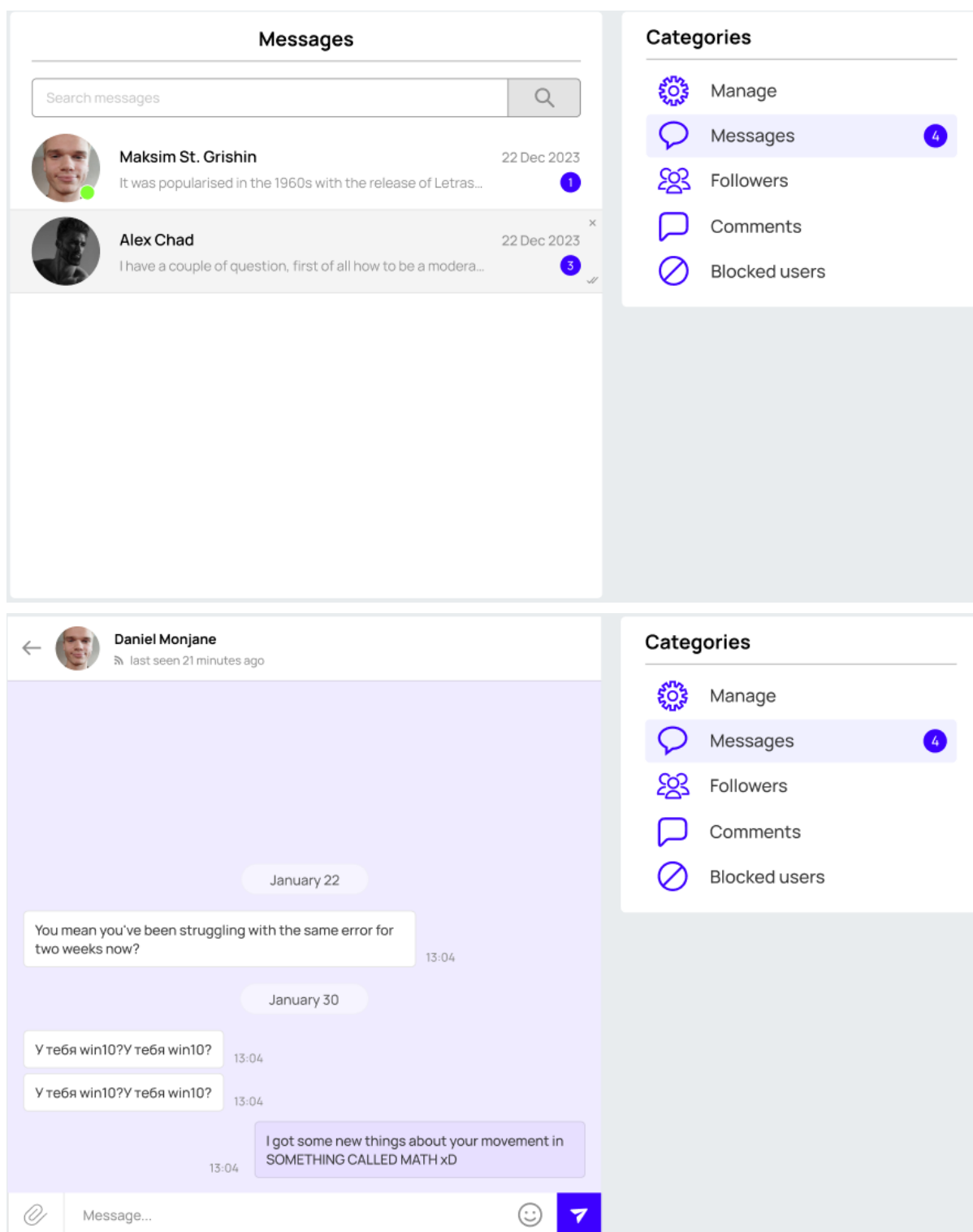
Если пользователь является модератором сообщества, то кнопка действия будет называться «модерировать», если админ, то «управлять». В случае, если админ нажмет на кнопку «управлять», откроется страница с категориями, по умолчанию стоит категория «управлять» (см. Рисунок 51).



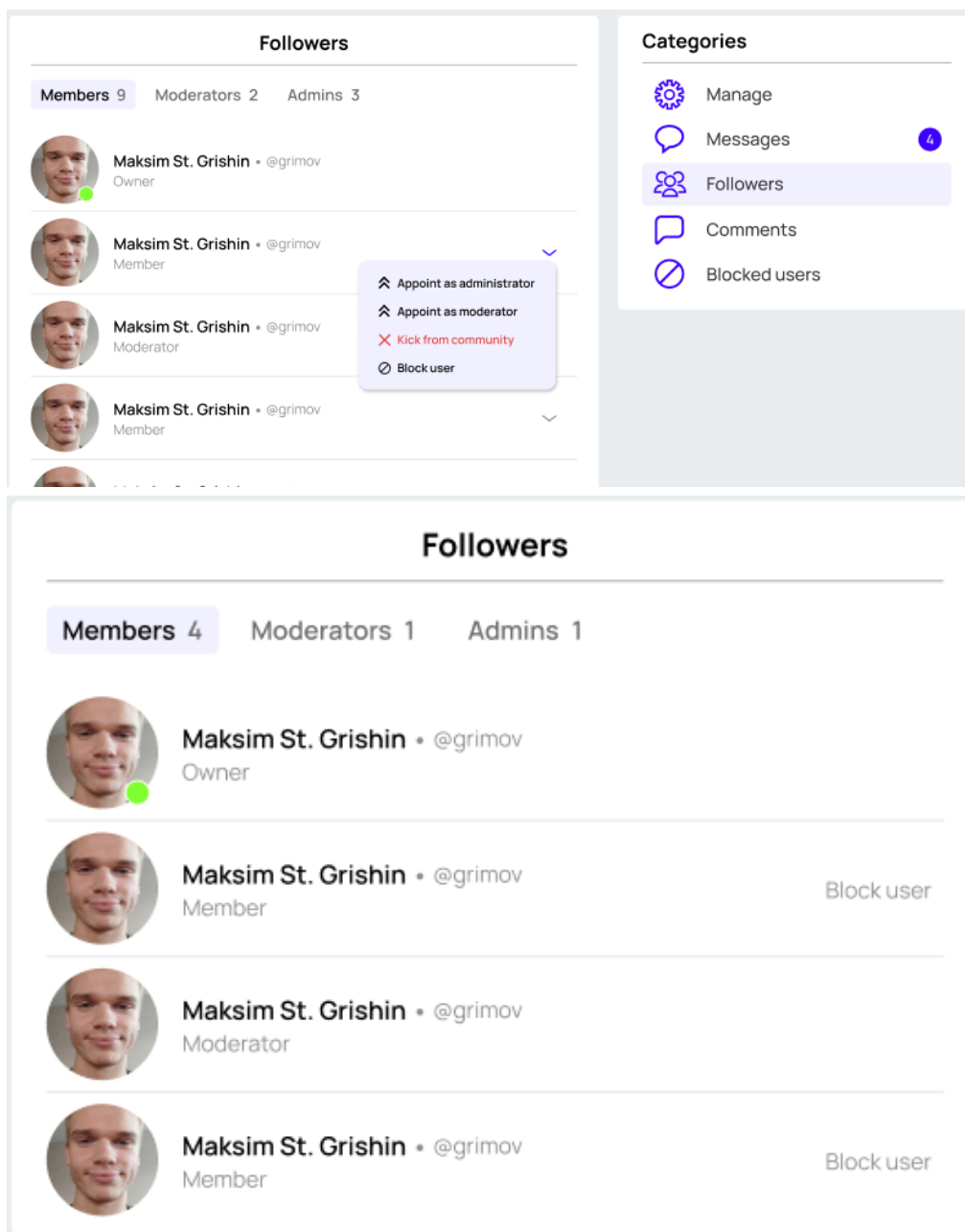
В категории управления можно менять название, никнейм, описание, тип приватности, тематика сообщества, а также можно отключить/включить комментарии на всех публикациях.

3.2.23 Сообщения сообщества

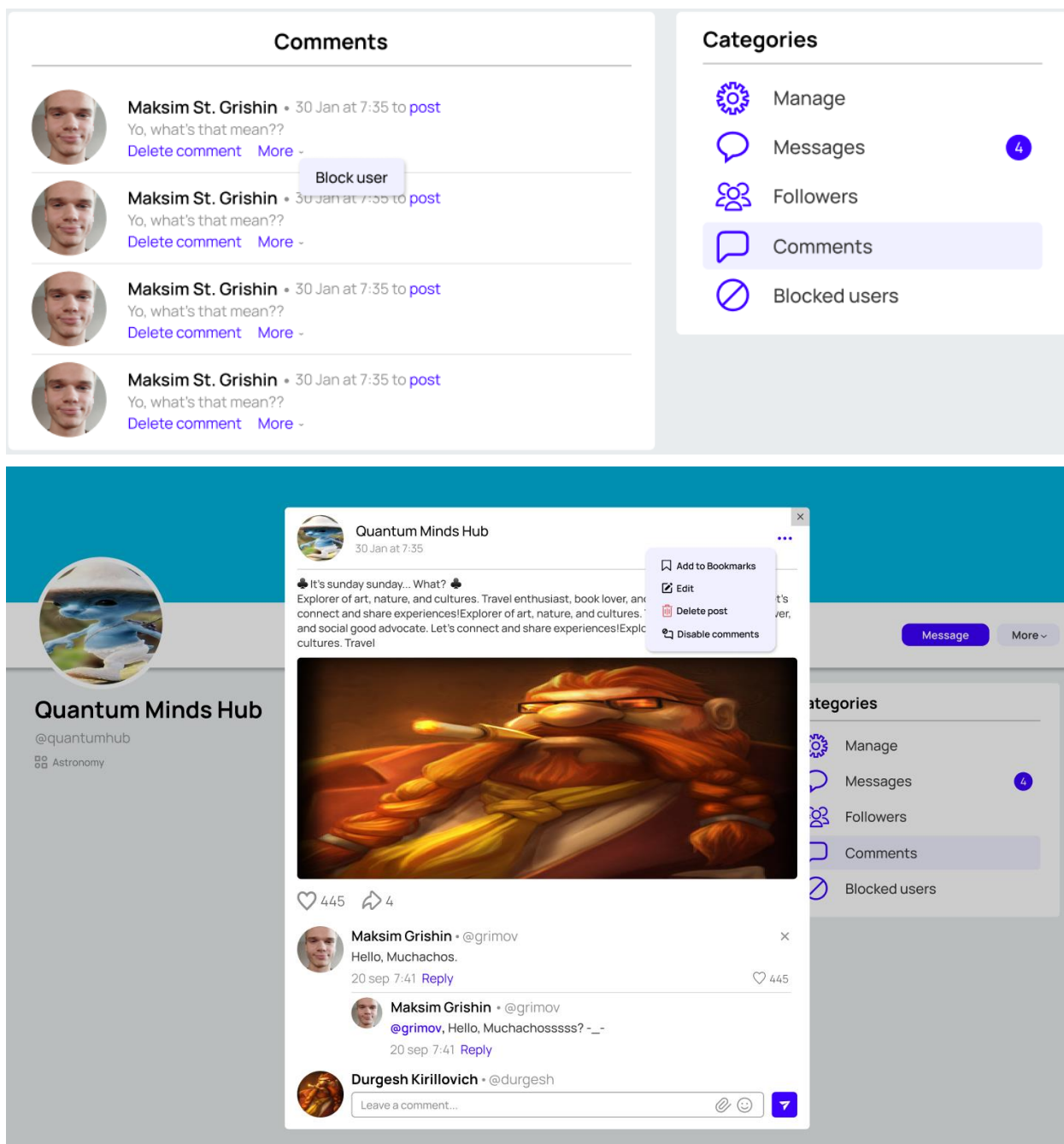
Если выбрана категория «сообщения», то откроется список чатов с пользователями (см. Рисунок 52), на них можно отвечать, открыв какой-либо чат (см. Рисунок 53), удалять чат, пометить как прочитанный.



Нажав на категорию «подписчики», откроется список всех участников сообщества (см. Рисунок 54). В списке есть фильтрация по роли (участник, модератор и администратор), создатель сообщества может изменять роли всех участников, блокировать и выгонять из сообщества. Модератор может блокировать участников (см. Рисунок 55).

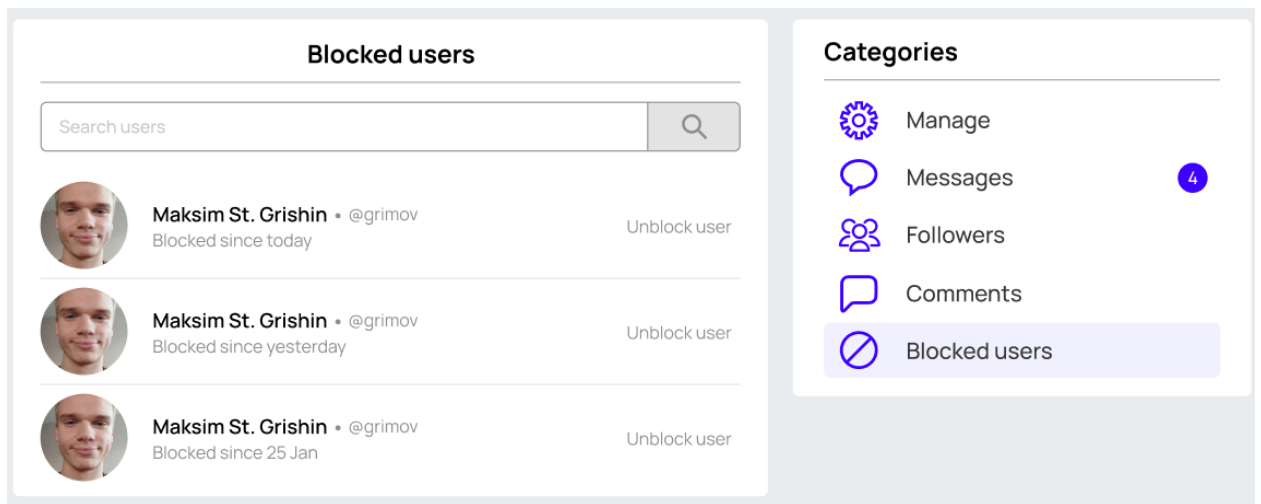


В категории «комментарии», администраторы и модераторы могут управлять комментариями под публикациями сообщества других пользователей. Отображается список всех комментариев, элемент с комментарием имеет краткую информацию об авторе, текст комментария, временная отметка, ссылка к связанной публикации и две кнопки действия. Первая кнопка – удалить комментарий, вторая – заблокировать пользователя (см. Рисунок 56). Нажав на ссылку с публикацией, откроется модальное окно с публикацией и всех информацией, связанной с ней (см. Рисунок 57).



3.2.24 Черный список сообщества

Также администратор может управлять черным списком сообщества, в котором находятся заблокированные пользователи (см. Рисунок 58). Администратор может разблокировать определенного пользователя, нажав на кнопку «разблокировать пользователя».



3.3. Backend разработка

В начале разработки бэкенда для социальной сети стояла задача выбора подходящего стека технологий, который бы обеспечил необходимую производительность, безопасность и масштабируемость. После анализа различных вариантов, было решено использовать сочетание Java и Spring Boot для серверной части, что позволило быстро приступить к разработке и обеспечить легкость в управлении зависимостями и конфигурациями.

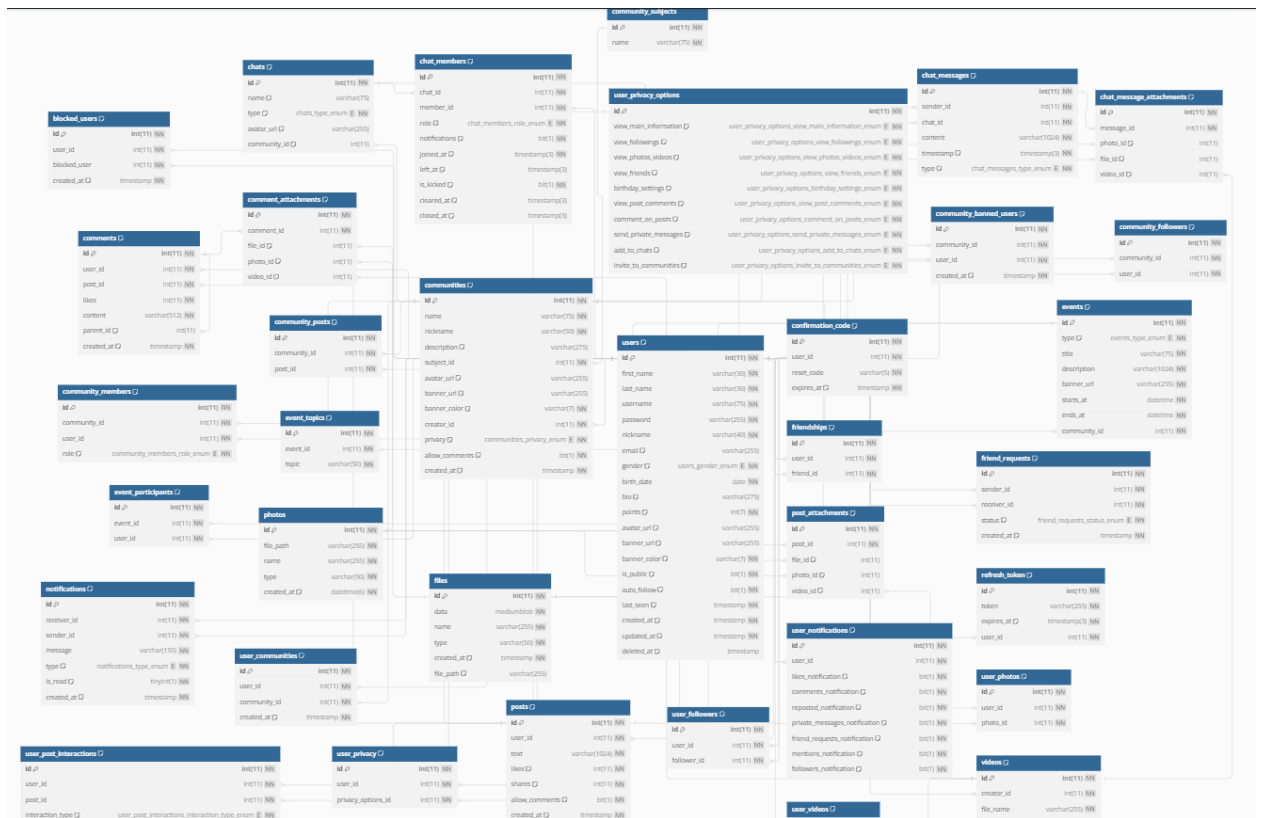
Для запуска проекта и его локальной разработки потребовалось настроить среду, которая включала в себя:

- Интегрированную среду разработки (IDE), поддерживающую Java и Spring Boot, для написания и тестирования кода.
- Систему управления базами данных, совместимую с Java и Spring Boot, для хранения и обработки данных.
- Инструменты для локального тестирования, такие как XAMPP, чтобы проверить работоспособность проекта перед его развертыванием.
- Контейнеризацию, например Docker, для обеспечения одинаковых условий разработки и продакшена и упрощения процесса развертывания.

С этими инструментами и технологиями команда смогла эффективно начать разработку бэкенда, обеспечивая стабильность и отзывчивость социальной сети. В последующих разделах будет представлено более детальное описание процесса разработки и использованных решений.

3.3.1 База данных

В основе любого веб-приложения социальной сети лежит база данных, которая является ключевым компонентом для обеспечения эффективной работы и взаимодействия пользователей. База данных Skill Shuffle спроектирована таким образом, чтобы обеспечить быстрый и надежный доступ к данным, а также их безопасность и целостность. Структура базы данных, отраженная на схеме (см. Рисунок 59), демонстрирует сложную систему связей между таблицами, которые предназначены для хранения информации о пользователях, их друзьях, сообщениях, группах, фотографиях и других важных аспектах социальных взаимодействий. Каждая таблица оптимизирована для выполнения конкретных задач, что позволяет обрабатывать большие объемы данных с высокой производительностью. Это обеспечивает стабильность и масштабируемость приложения, позволяя ему расти и развиваться вместе с увеличением числа пользователей и их активности.



3.3.2 Подключение к базе данных

В серверной части использовался Spring Boot для создания стандартизированного и удобного в эксплуатации исполнительного окружения. Для взаимодействия с базой данных применялся Java Persistence API (JPA), который служит мостом между объектно-

ориентированным кодом Java и реляционной базой данных. Hibernate был выбран в качестве провайдера JPA для упрощения процесса маппинга объектов Java в таблицы базы данных и обратно, что позволило разработчикам сосредоточиться на бизнес-логике, минимизируя необходимость написания SQL-запросов (см. Рисунок 60 и 61).

```
@Data
@Entity
@Builder
@AllArgsConstructor
@NoArgsConstructor
@Table(name = "chat_messages")
public class ChatMessage {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "sender_id", nullable = false)
    private User sender;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "chat_id", nullable = false)
    private Chat chat;

    @Column(name = "content", nullable = false, length = 1024)
    private String content;

    @Column(name = "timestamp", nullable = false)
    private Timestamp timestamp;

    @Enumerated(EnumType.STRING)
    @Column(name = "type", nullable = false)
    private MessageType type;

    @JsonIgnore
    public boolean isMessage() {
        return this.type == MessageType.MESSAGE;
    }

    @JsonIgnore
    public boolean isAnnouncement() {
        return this.type == MessageType.ANNOUNCEMENT;
    }
}
```

```
public interface ChatMemberRepository extends JpaRepository<ChatMember, Integer> {
    List<ChatMember> findAllByChatId(Integer chatId);
    void deleteAllByChatId(Integer id);
    ChatMember findChatMemberByChatIdAndMemberId(Integer id, Integer id1);
    ChatMember findFirstByChatIdAndMemberUsername(Integer id, String username);
    ChatMember findFirstByChatIdAndRole(Integer id, MemberRole memberRole);
}
```

В процессе разработки, разработчики использовали локальную базу данных для тестирования и отладки функциональности приложения. Это позволило быстро вносить изменения и проверять их в реальном времени без необходимости подключения к внешним сервисам. При переходе к стадии продакшн, для хранения данных выбран от Amazon - Relational Database Service (RDS). AWS RDS предоставляет масштабируемые и управляемые сервисы баз данных, которые обеспечивают высокую доступность,

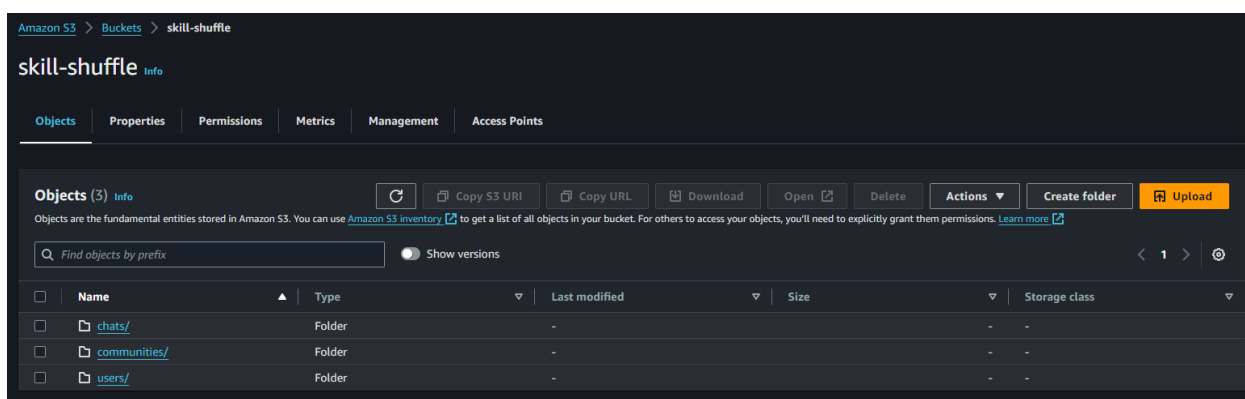
надежность и безопасность. Использование AWS RDS в развернутом приложении позволяет оптимизировать ресурсы и гарантировать бесперебойную работу сервиса для наших пользователей. Ниже приведен пример файл конфигурации подключения к базе (см. Рисунок 62).

```
# Database connection configuration (RDS)
#spring.datasource.url=jdbc:mariadb://
#spring.datasource.username=
#spring.datasource.password=
#spring.jpa.hibernate.ddl-auto=update

spring.datasource.url=jdbc:mariadb://localhost:3306/skill_shuffle
spring.datasource.username=root
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=update
```

3.3.3 Хранение статичных файлов в AWS S3

Статичные файлы, такие как изображения и видео, загруженные пользователями, сохраняются в облачном хранилище Amazon Web Services (AWS) S3 (см. Рисунок 63). Это обеспечивает высокую доступность и безопасность данных. Ссылки на эти файлы хранятся в базе данных, что позволяет легко интегрировать их в приложение и предоставлять доступ пользователям. Использование AWS S3 способствует масштабируемости хранилища и оптимизации затрат, так как оплата происходит только за фактически использованный объем данных.



3.3.4 Маршрутизация

Этот фрагмент кода (см. рисунок 64) конфигурирует безопасность веб-приложения. Он определяет, какие URL-адреса могут быть доступны без аутентификации и какие требуют

подтверждения личности пользователя. В данном случае, все запросы к адресам, начинающимся с /auth/, к вебсокету /ws, и к статическим ресурсам /static/ разрешены без аутентификации. Любой другой запрос должен проходить проверку на аутентификацию.

Для этого используется JwtAuthenticationFilter, который проверяет наличие и валидность токена в запросе. Если токен отсутствует или недействителен, JwtAuthenticationEntryPoint перехватывает запрос и возвращает ошибку аутентификации.

Эта конфигурация обеспечивает гибкость в управлении доступом к ресурсам приложения и позволяет легко интегрировать системы аутентификации на основе токенов.

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Autowired
    private JwtAuthenticationEntryPoint point;

    @Autowired
    private JwtAuthenticationFilter filter;

    private final String[] allowedUrls = new String[] {
        "/auth/*",
        "/ws",
        "/static/**"
    };

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity httpSecurity) throws Exception {
        httpSecurity
            .authorizeHttpRequests(authorizeRequests ->
                authorizeRequests
                    .requestMatchers(allowedUrls).permitAll()
                    .anyRequest().authenticated()
            )
            .csrf(AbstractHttpConfigurer::disable)
            .exceptionHandling(e -> e.authenticationEntryPoint(this.point))
            .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .addFilterBefore(filter, UsernamePasswordAuthenticationFilter.class);
        return httpSecurity.build();
    }
}
```

Представленные методы (см. Рисунок 65 и 66) являются частью бэкенда веб-приложения и обеспечивают важные функции для управления учетными записями пользователей и взаимодействия в чатах. Они используют REST API для обработки HTTP-запросов и взаимодействия с базой данных и другими сервисами приложения.

```

// Update chat avatar
@PatchMapping("/chats/{id}/avatar")
public ResponseEntity<Chat> updateChatAvatar(@PathVariable("id") Integer id,
                                             @RequestParam("avatarBlob") MultipartFile avatarBlob) {

    Chat chat = chatRepository.findChatById(id);
    Chat updatedChat = chatService.updateChatAvatar(chat, avatarBlob);
    if (updatedChat != null) {
        return ResponseEntity.ok(updatedChat);
    } else {
        return ResponseEntity.internalServerError().build();
    }
}

// Get chat messages
@GetMapping("/chats/{id}/messages")
public List<MessageDTO> getChatMessages(@PathVariable("id") Integer id,
                                         @RequestParam(defaultValue = "30") int limit,
                                         @RequestParam(defaultValue = "0") int offset) {

    ChatMember member = chatMemberRepository.findChatMemberByChatIdAndMemberId(id, userService.getCurrentUser().getId());
    return messageService.getChatMessages(id, member, limit, offset);
}

// Invite members to chat
@PostMapping("/chats/{id}/members")
public ChatDTO addChatMembers(@PathVariable("id") Integer id,
                              @RequestBody List<String> users) {

    Chat chat = chatRepository.findChatById(id);
    return chatService.inviteMembersToChat(chat, users);
}

// Get friend list for a chat 'add members' modal
@GetMapping("/chats/{id}/friends")
public List<PublicUserDTO> getFriendListForChat(@PathVariable("id") Integer id) {

    Chat chat = chatRepository.findChatById(id);
    return chatService.getFriendListForChat(chat);
}

```

```

public class AuthController {

    @Autowired
    private UserService userService;

    @PostMapping("/login")
    public ResponseEntity<> login(@RequestBody AuthRequestDTO request, HttpServletResponse response) {
        // Authenticate user in the spring security context
        try {
            userService.doAuthenticate(request.getUsername(), request.getPassword());
        } catch (Exception e) {
            return new ResponseEntity<>("Invalid credentials", HttpStatus.UNAUTHORIZED);
        }

        // Generate access token and refresh token
        JwtResponseDTO jwtResponse = userService.authenticateUser(request, response);
        if (jwtResponse != null) {
            return new ResponseEntity<>(jwtResponse, HttpStatus.OK);
        }
        return new ResponseEntity<>(Map.of("message", "User not found"), HttpStatus.NOT_FOUND);
    }

    @PostMapping("/register")
    public ResponseEntity<> register(@RequestPart("user") String userStr,
                                    @RequestPart(value = "avatarBlob", required = false) MultipartFile avatarBlob) throws JsonProcessingException {

        ObjectMapper mapper = new ObjectMapper();
        User registrationUser = mapper.readValue(userStr, User.class);

        if (userService.checkNicknameDuplicate(registrationUser.getNickname())) {
            return new ResponseEntity<>(Map.of("message", "Nickname already in use", "reason", "nickname"), HttpStatus.CONFLICT);
        }

        if (userService.checkUserDuplicate(registrationUser.getUsername(), registrationUser.getEmail())) {
            return new ResponseEntity<>(Map.of("message", "User already exists", "reason", "credentials"), HttpStatus.CONFLICT);
        }

        User user = userService.saveUser(registrationUser, avatarBlob);
        if (user != null) {
            return new ResponseEntity<>(user, HttpStatus.CREATED);
        }

        return new ResponseEntity<>("Error occurred while registering user", HttpStatus.BAD_REQUEST);
    }
}

```

3.3.5 Функции

В разработке серверной части затрагиваются важные аспекты, которые обеспечивают эффективную работу веб-приложения. Особое внимание уделяется механизмам, которые позволяют пользователям безопасно взаимодействовать с системой в реальном времени. Примером такой функциональности является компонент `AuthChannelInterceptorAdapter` (см. Рисунок 67), который отвечает за аутентификацию пользователей при установлении соединения с веб-сокет протоколом поверх TCP, подписке на конечные точки протокола и отписке от них. Этот компонент гарантирует, что каждое действие пользователя проходит проверку безопасности, что критически важно для поддержания целостности и конфиденциальности данных в приложении.

```
public class AuthChannelInterceptorAdapter implements ChannelInterceptor {  
    @Override  
    public Message<?> preSend(@NonNull Message<?> message, @NonNull MessageChannel channel) throws AuthenticationException {  
        StompHeaderAccessor accessor = MessageHeaderAccessor.getAccessor(message, StompHeaderAccessor.class);  
  
        // Handle user connection  
        if (StompCommand.CONNECT == Objects.requireNonNull(accessor).getCommand()) {  
            // Get token from the header and remove the prefix  
            String token = accessor.getFirstNativeHeader(AUTHORIZATION_HEADER);  
            if (token != null && token.startsWith(AUTHORIZATION_HEADER_PREFIX)) {  
                token = token.substring(AUTHORIZATION_HEADER_PREFIX.length());  
            }  
  
            UsernamePasswordAuthenticationToken user = webSocketAuthenticatorService.getAuthenticatedOrFail(token);  
            accessor.setUser(user);  
        }  
  
        // Handle user subscription  
        if (StompCommand.SUBSCRIBE == Objects.requireNonNull(accessor).getCommand()) {  
            String username = Objects.requireNonNull(accessor.getUser()).getName();  
            String destination = accessor.getDestination();  
            String subscriptionId = accessor.getSubscriptionId();  
            if (username != null && destination != null && subscriptionId != null) {  
                sessionService.addSession(username, destination, subscriptionId);  
  
                // Add previous chat ID to the user's session  
                if (destination.startsWith("/user/chat/")) {  
                    String chatId = destination.split("/")[3];  
                    sessionService.addSession(username, "prev-chat", chatId);  
                }  
            }  
        }  
  
        // Handle user unsubscription  
        if (StompCommand.UNSUBSCRIBE == Objects.requireNonNull(accessor).getCommand()) {  
            String username = Objects.requireNonNull(accessor.getUser()).getName();  
            String subscriptionId = accessor.getSubscriptionId();  
            if (username != null && subscriptionId != null) {  
                sessionService.removeSession(username, subscriptionId);  
            }  
        }  
    }  
}
```

В приложении была реализована система уведомлений (см. Рисунок 68 и 69), которая играет ключевую роль в поддержании активности и вовлеченности пользователей. Одним из основных типов уведомлений являются оповещения о новых сообщениях. Эта функция автоматически уведомляет пользователей, когда они получают новые сообщения в чатах (которые не заглушены пользователем), обеспечивая тем самым, что ни одно важное сообщение не останется без внимания. Такие уведомления не только повышают удобство использования приложения, но и способствуют более быстрой и эффективной коммуникации между пользователями.

```
// Create new message notification based on chat type
public ChatNotificationDTO generateMessageNotification(Integer chatId, ChatMessage message, PublicUserDTO sender) {
    String notificationMessage;
    Chat chat = chatRepository.findChatById(chatId);

    if (chat.isPrivate()) {
        notificationMessage = sender.getFirstName() + " sent you a message";
    } else {
        notificationMessage = "New message in " + chat.getName();
    }

    // Return notification object
    return new ChatNotificationDTO(
        chat,
        sender,
        notificationMessage,
        message.getContent(),
        NotificationType.CHAT_MESSAGE
    );
}
```

```
private void returnMessageToUsers(ChatMessage message, User sender) {
    Integer chatId = message.getChat().getId();
    List<String> usernames = userService.getUsernamesInChat(chatId);

    for (String username : usernames) {
        if (sessionService.isConnectionActive(username)) {
            // Send the message to all users who subscribed to chat
            if (sessionService.isUserSubscribed(username, "/user/chat/" + chatId)) {
                messagingTemplate.convertAndSendToUser(username, "/chat/" + chatId, convertToDTO(message));
                continue;
            }

            // Generate notification for the user
            ChatNotificationDTO notification = generateMessageNotification(chatId, message, userService.getPublicUserDTO(sender));

            // Send new message notification to the user, for the chat list
            if (sessionService.isUserSubscribed(username, "/user/chat")) {
                Object[] notificationAndMessage = {notification, convertToDTO(message)};
                messagingTemplate.convertAndSendToUser(username, "/chat", notificationAndMessage);
                continue;
            }

            // Send notification to user who is currently not subscribed to any chat endpoint
            ChatMember chatMember = chatMemberRepository.findChatMemberByChatIdAndMemberId(chatId, userRepository.findByUsername(username).getId());
            if (chatMember.hasNotifications()) {
                messagingTemplate.convertAndSendToUser(username, "/notification", notification);
            }
        }
    }
}
```

Получения информации об авторизованном пользователе происходит через контекст безопасности Spring Security. Когда пользователь проходит аутентификацию, его данные сохраняются в контексте, который доступен во всем приложении. Метод `getCurrentUser` проверяет этот контекст и, если в нем есть информация о пользователе, которая не относится к анонимному токenu, извлекает имя пользователя (см. Рисунок 70 и 71). Затем, используя это имя, метод обращается к репозиторию для получения полных данных пользователя из базы данных. Если аутентификация не была пройдена, метод вернет `null`, указывая на отсутствие авторизованного пользователя. Это позволяет приложению идентифицировать пользователя, который выполняет запрос, и предоставлять ему соответствующие данные и функции.

```
public User getCurrentUser() {
    Authentication authentication = SecurityContextHolder.getContext().getAuthentication();
    if (authentication != null && !(authentication instanceof AnonymousAuthenticationToken)) {
        String username = authentication.getName();
        return userRepo.findByUsername(username);
    }
    return null;
}

package com.dzalex.skillshuffle.repositories;

import com.dzalex.skillshuffle.entities.User;
import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository extends JpaRepository<User, Integer> {
    User findByNickname(String nickname);
    User findByUsername(String username);
    User findByEmail(String email);
}
```

3.3.6 Функции авторизации и регистрации

После того как пользователь заполнил поля для входа, если данные верны, то создается токен доступа (`access token`), который действует два часа и хранится в `httpOnly cookie`. Этот токен нужен для того, чтобы иметь доступ к сайту как аутентифицированный пользователь. Если пользователь также нажал кнопку «Запомнить меня» в форме входа, также параллельно создается токен продления (`refresh token`), он действует уже 7 дней, хранится он не только в `httpOnly cookie`, но и в базе данных. С помощью этого токена можно заходить на сайт на протяжении недели не вводя данные в форму входа при каждом посещении сайта. Соответственно если срок действия токена доступа истек, то при наличии `refresh token` генерируется новый `access token` (см. Рисунок 72).

```

public JwtResponseDTO authenticateUser(AuthRequestDTO request, HttpServletResponse response) {
    UserDetails userDetails = userDetailsService.loadUserByUsername(request.getUsername());
    if (userDetails != null) {
        // If user clicked remember me, create refresh token and save it in the database and in the cookie
        if (request.isRememberMe()) {
            RefreshToken refreshToken = refreshTokenService.generateRefreshToken(request.getUsername());
            helper.createRefreshTokenCookie(response, refreshToken);
        }

        // Create access token and save it in the cookie
        String accessToken = helper.createAccessTokenCookie(response, userDetails);

        User user = userRepo.findByUsername(request.getUsername());
        return JwtResponseDTO.builder()
            .username(request.getUsername())
            .accessToken(accessToken)
            .user(getPublicUserDTO(user))
            .build();
    }
    return null;
}

```

Процесс регистрации в приложении включает в себя несколько ключевых шагов, обеспечивающих безопасность и удобство для пользователя. При регистрации нового пользователя его пароль шифруется с использованием надежного алгоритма кодирования, что предотвращает возможность чтения пароля в явном виде. Затем информация о пользователе сохраняется в базе данных. Если пользователь загрузил аватар, то он также обрабатывается и сохраняется в файловой системе или облачном хранилище, а ссылка на изображение добавляется в профиль пользователя (см. Рисунок 73).

Класс User представляет собой модель данных пользователя (см. Рисунок 74), включая такие поля, как имя, фамилия, имя пользователя, пароль, никнейм, электронная почта, пол, дата рождения, биография, очки, URL аватара и баннера, настройки приватности, а также временные метки последнего посещения, создания и обновления профиля. Эта модель используется для хранения и управления данными пользователя в базе данных.

```

public User saveUser(User user, MultipartFile avatarBlob) {
    user.setPassword(passwordEncoder().encode(user.getPassword()));
    User savedUser = userRepo.save(user);
    User updatedUser = saveAvatar(savedUser, avatarBlob);
    return updatedUser != null ? updatedUser : savedUser;
}

private User saveAvatar(User user, MultipartFile avatarBlob) {
    if (avatarBlob != null) {
        String avatarFilePath = "users/user-" + user.getId() + "/avatar/";
        String avatarUrl = fileService.uploadFile(avatarBlob, avatarFilePath);
        if (avatarUrl != null) {
            user.setAvatarUrl(avatarUrl);
            return userRepo.save(user);
        }
    }
    return null;
}

```

```

@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Column(name = "first_name", nullable = false, length = 30)
    private String firstName;

    @Column(name = "last_name", nullable = false, length = 30)
    private String lastName;

    @Column(name = "username", nullable = false, length = 75)
    private String username;

    @Column(name = "password", nullable = false)
    private String password;

    @Column(name = "nickname", nullable = false, length = 40)
    private String nickname;

    @Column(name = "email")
    private String email;

    @Enumerated(EnumType.STRING)
    @Column(name = "gender", nullable = false)
    private Gender gender;

    @Column(name = "birth_date")
    private Date birthDate;

    @Column(name = "bio", length = 275)
    private String bio;

    @Builder.Default
    @Column(name = "profile_picture", nullable = false)

```

4. ТЕСТИРОВАНИЕ ПРОЕКТА И АНАЛИЗ РЕЗУЛЬТАТОВ

В процессе разработки применялись различные методы тестирования, включая модульное, интеграционное, системное и приемочное, чтобы обеспечить соответствие продукта установленным требованиям и ожиданиям пользователей. Результаты тестирования были тщательно проанализированы, что позволило выявить и устранить недочеты, повысить стабильность и улучшить общую производительность приложения. Этот этап разработки играет критически важную роль в обеспечении надежности и безопасности продукта перед его запуском.

4.1. Выбор методов тестирования

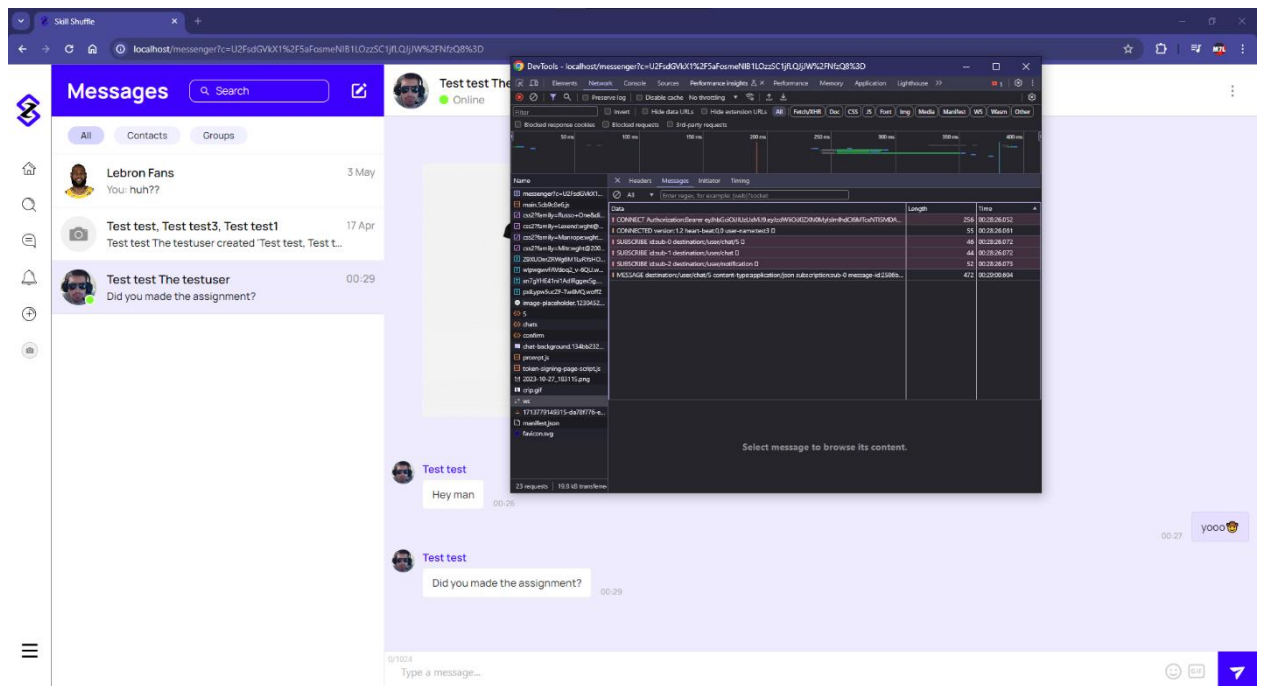
Для тестирования проекта были выбраны методы, ориентированные как на отдельные модули, так и на весь проект в целом. Тестирование белым ящиком было использовано для проверки внутренних структур и алгоритмов работы приложения, что позволило выявить и устранить потенциальные проблемы на ранних этапах разработки. Тестирование серым ящиком, сочетающее элементы тестирования белым и черным ящиками, применялось для проверки интеграции различных компонентов системы.

4.2. Тестирование

Тестирование включало в себя использование Selenium для автоматизации действий пользователя и проверки пользовательского интерфейса, IntelliJ Debugger для отладки кода и выявления ошибок в логике, а также Postman для тестирования API и взаимодействия с сервером. Эти инструменты помогли обеспечить высокое качество кода и удобство использования сайта.

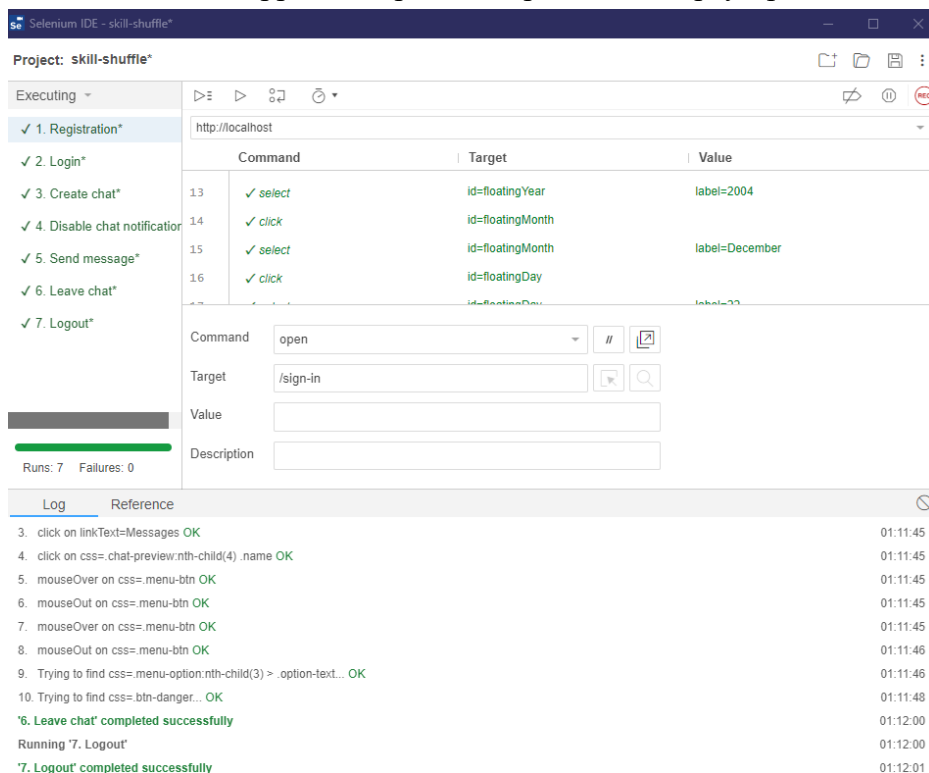
4.2.1 Ручное тестирование

В проекте было проведено ручное тестирование (см. Рисунок 75), которое позволило тестировщикам лично оценить функциональность приложения и выявить ошибки. Этот метод не требует автоматизации и предоставляет возможность непосредственного взаимодействия с программным обеспечением.



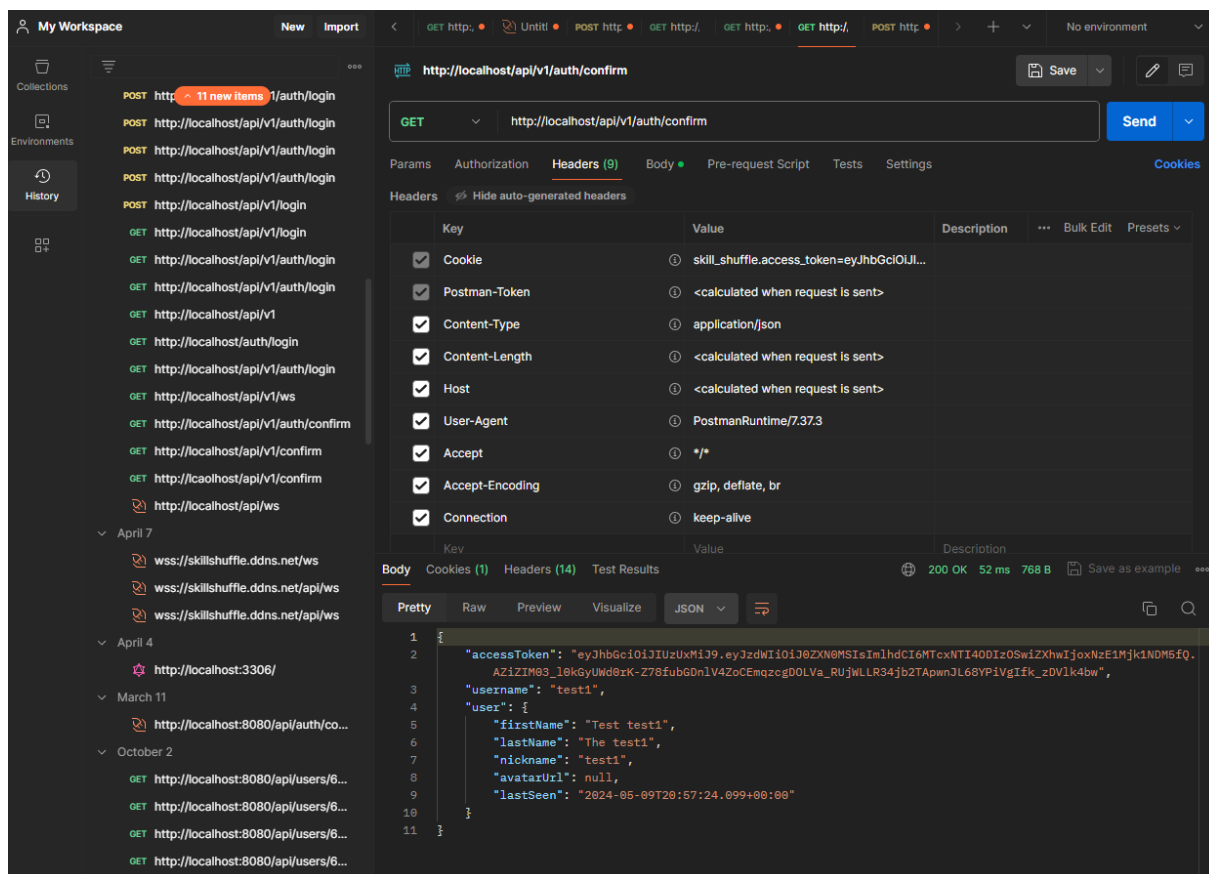
4.2.2 Тестирование с помощью Selenium

В рамках проекта было проведено тестирование с использованием Selenium (см. Рисунок 76), инструмента для автоматизации браузеров. Это позволило разработчикам эффективно имитировать действия пользователя и проверять функциональность веб-интерфейса. Selenium был особенно полезен для проверки сложных взаимодействий на веб-страницах и обеспечения их корректной работы в различных браузерах.



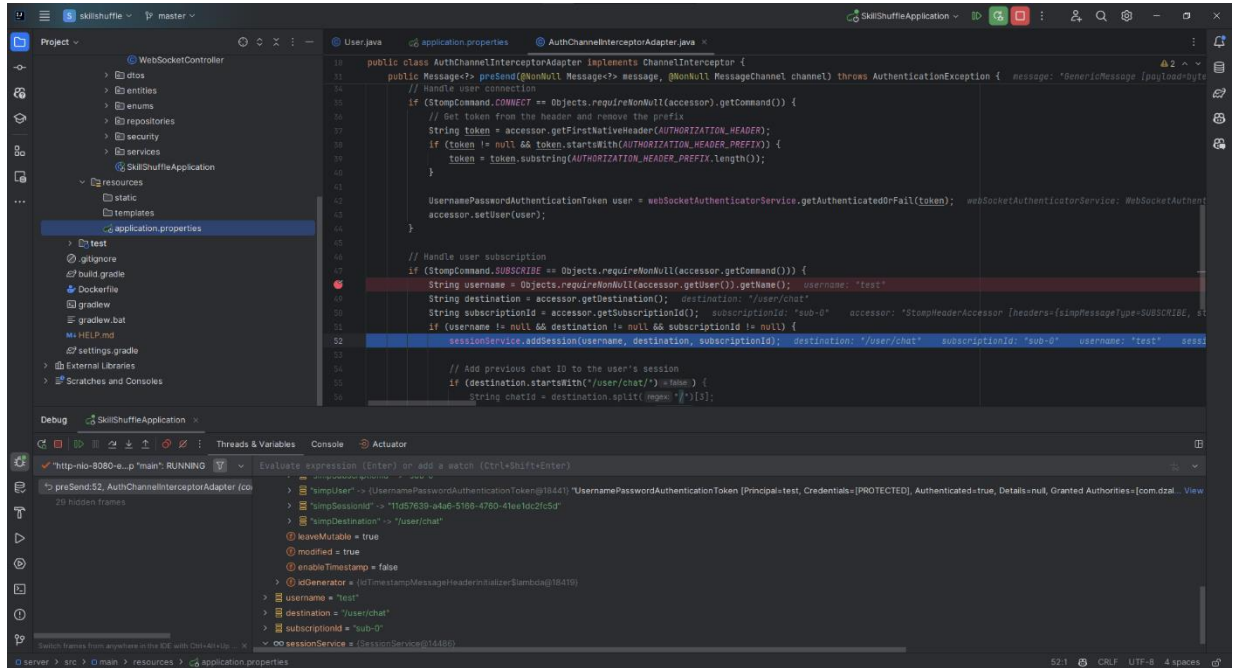
4.2.3 Тестирование с помощью Postman

Разработчики использовали Postman для тестирования API (см. Рисунок 77), что позволило им автоматизировать отправку запросов и анализ ответов сервера. Этот инструмент оказался неоценимым для проверки корректности работы API и обеспечения его надежности.



4.2.4 Отладка с помощью IntelliJ Debugger

Отладка кода проекта осуществлялась с помощью IntelliJ Debugger (см. Рисунок 78). Этот инструмент позволил разработчикам пошагово проследить выполнение программы, оценить состояние переменных и точно определить местоположение ошибок.



4.3. Анализ результатов

4.4. Возможности дальнейшего развития

ЗАКЛЮЧЕНИЕ

Социальная сеть «Skill Shuffle» нацелена на объединение людей ради совместного изучения, качественный и удобный обмен информацией. Пользователи, создавшие аккаунт, имеют возможность полноценно коммуницировать с другими людьми и пользоваться сетью, притом имея удобные инструменты для быстрого и простого взаимодействия, например:

- уведомления;
- чат;
- навигация;
- поиск;
- различные настройки.

«Skill Shuffle» соединяет в себе социальную сеть и элементы форума, которые будут добавлены в будущем, благодаря которым пользователи смогут заниматься саморазвитием, создавать сообщества, обмениваться информацией, задавать вопросы и получать ответы, устраивать семинары.

Авторы старались сделать дизайн разгруженным и интуитивно понятным, благодаря чему сайт будет удобен для большого числа пользователей. А также была проделана большая работа над оптимизацией проекта, поскольку в социальных сетях очень важна скорость загрузки.

ЛИТЕРАТУРА

1. What is Amazon EC2? - Amazon Elastic Compute Cloud [Интернет] Available: <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html> [Использовано: 06.04.2024]
2. What is Amazon Relational Database Service (Amazon RDS) [Интернет] Available: <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Welcome.html> [Использовано: 06.04.2024]
3. What is Amazon Elastic Container Registry? - Amazon ECR [Интернет] Available: <https://docs.aws.amazon.com/AmazonECR/latest/userguide/what-is-ecr.html> [Использовано: 06.04.2024]
4. What is Amazon CloudWatch? - Amazon CloudWatch [Интернет] Available: <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/WhatIsCloudWatch.html> [Использовано: 06.04.2024]
5. What is Amazon S3? - Amazon Simple Storage Service [Интернет] Available: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html> [Использовано 06.04.2024]
6. React Bootstrap – Introduction [Интернет] Available: <https://react-bootstrap.netlify.app/docs/getting-started/introduction> [Использовано: 11.04.2024]
7. Spring Boot – Overview [Интернет] Available: <https://docs.spring.io/spring-boot/index.html> [Использовано: 11.04.2024]
8. Docker – Manuals [Интернет] Available: <https://docs.docker.com/manuals/> [Использовано: 11.04.2024]
9. Using WebSocket to build an interactive web application [Интернет] Available: <https://spring.io/guides/gs/messaging-stomp-websocket> [Использовано: 15.04.2024]
10. FullCalendar – React Component [Интернет] Available: <https://fullcalendar.io/docs/react> [Использовано: 03.05.2024]

5. ПРИЛОЖЕНИЯ

5.1. Список иллюстраций